GERAÇÃO DE LAYOUTS DE FASES 3D ASSISTIDA POR MODELOS DE LINGUAGEM DE GRANDE PORTE

Gustavo Rafael Nunes DURÃES[1,*], Wagner Ferreira de BARROS[2]
[1]Discente do Curso de Bach. em Ciência da Computação, IFNMG — Campus Montes Claros.
grnd@aluno.ifnmg.edu.br | **autor correspondente*
[2]Docente do IFNMG — Campus Montes Claros. wagner.barros@ifnmg.edu.br

RESUMO

A criação manual de fases para jogos digitais é um processo exaustivo que exige a definição minuciosa de caminhos, salas, regras de progressão e a validação geométrica de espaços tridimensionais (3D). Embora os Modelos de Linguagem de Grande Porte (LLMs) apresentem excelente capacidade de interpretação semântica e planejamento lógico a partir de linguagem natural, eles demonstram baixa confiabilidade no raciocínio espacial e geométrico exigido para a modelagem 3D. O presente trabalho propõe uma arquitetura que separa a geração procedimental de fases em duas responsabilidades distintas: um modelo atuando como "arquiteto", encarregado de traduzir intenções textuais do usuário em um grafo lógico estruturado (JSON), e um algoritmo determinístico atuando como "engenheiro", desenvolvido na *engine* Godot, encarregado de validar a acessibilidade das salas e instanciar os componentes 3D no espaço sem colisões indevidas. O objetivo da pesquisa é avaliar o impacto da etapa de validação determinística na coerência das fases geradas, comparando a abordagem assistida estruturalmente com abordagens de geração unificada. Como resultados esperados, almeja-se a criação de um *addon* funcional para Godot que demonstre que a divisão de tarefas (lógica vs. geometria) supera gargalos de inconsistência arquitetural presentes nas abordagens baseadas puramente em LLMs, garantindo a geração de fases solváveis, logicamente conectadas e geometricamente precisas.

Palavras-chave: Geração Procedural de Conteúdo. Modelos de Linguagem de Grande Porte. Godot Engine. Level Design. JSON.

ABSTRACT

The manual creation of levels for digital games is an exhausting process that requires the meticulous definition of paths, rooms, progression rules, and the geometric validation of three-dimensional (3D) spaces. Although Large Language Models (LLMs) exhibit excellent semantic interpretation and logical planning capabilities from natural language, they show low reliability in the spatial and geometric reasoning required for 3D modeling. This ongoing work proposes an architecture that separates procedural level generation into two distinct responsibilities: a model acting as an "architect", tasked with translating the user's textual intentions into a structured logical graph (JSON), and a deterministic algorithm acting as an "engineer", developed in the Godot engine, tasked with validating

room accessibility and instantiating 3D components in space without improper collisions. The aim of this research is to evaluate the impact of the deterministic validation step on the coherence of the generated levels, comparing the structurally assisted approach with unified generation approaches. As expected results, we aim to create a functional addon for Godot that demonstrates that dividing tasks (logic vs. geometry) overcomes architectural inconsistency bottlenecks present in purely LLM-based approaches, ensuring the generation of solvable, logically connected, and geometrically accurate levels.

Keywords: Procedural Content Generation. Large Language Models. Godot Engine. Level Design. JSON.

---

1 Introdução

A Geração Procedural de Conteúdo (PCG - *Procedural Content Generation*) é amplamente utilizada na indústria de jogos para criar mundos expansivos e variados de forma algorítmica, mitigando o esforço manual do *Level Design* (SUMMERVILLE et al., 2018). Tradicionalmente, métodos determinísticos garantem que o conteúdo gerado (como masmorras ou mapas) atenda a critérios estritos de jogabilidade, como conectividade e solvabilidade.

Com o advento dos Modelos de Linguagem de Grande Porte (LLMs), surgiu um novo paradigma denominado PCG via Machine Learning (PCGML). Trabalhos recentes demonstraram a viabilidade de usar LLMs para gerar níveis estruturados em representações textuais bidimensionais, como na matriz de posições de blocos para *Sokoban* (TODD et al., 2023). Ademais, abordagens integradas com motores de jogos demonstram o alto potencial dessas redes na orquestração semântica e na assistência criativa contínua (TANG et al., 2025).

Apesar da alta proficiência em raciocínio lógico e interpretação de intenções em linguagem natural, os LLMs lutam significativamente na coordenação espacial e raciocínio puramente geométrico em espaços tridimensionais (3D). Ao tentar gerar o posicionamento 3D absoluto de estruturas arquitetônicas complexas através de uma rede neural estocástica, frequentemente surgem erros críticos como intersecção de malhas, corredores sem saída e chaves geradas em salas inacessíveis (XU; VERBRUGGE, 2025b; SUN et al., 2025).

O objetivo principal deste estudo é desenvolver e avaliar um sistema no qual o LLM processa o *prompt* do usuário e gera exclusivamente a macroestrutura lógica da fase, representada através de um grafo em formato JSON, propondo uma abordagem arquitetural de responsabilidades segregadas. A premissa fundamental, que atua como problema motivador da pesquisa, é a de que o LLM pensa e planeja a fase conceitualmente, enquanto um algoritmo determinístico a verifica e a constrói fisicamente. A instânciação geométrica e a validação topológica, por sua vez, são delegadas a um módulo procedimental desenvolvido na *engine* Godot.

Espera-se que essa segregação reduza o escopo de falha do modelo probabilístico, mantendo a expressividade oriunda da linguagem natural e assegurando a criação de mapas totalmente jogáveis.

2 Material e Métodos

Para operacionalizar a geração assistida, a metodologia adota um fluxo de trabalho em *pipeline*, dividido em três etapas sequenciais: (1) Processamento da Linguagem, (2) Validação Determinística e (3) Construção e Testes, todas integradas através de um *addon* em desenvolvimento para o Godot Engine.

1. Processamento da Linguagem: Inicialmente, um LLM recebe a requisição estrutural do desenvolvedor/jogador em linguagem natural (e.g., 'crie uma masmorra com uma sala inicial, uma bifurcação onde um lado contém um tesouro e o outro lado demanda uma chave localizada no tesouro para acessar a sala do chefe"). O modelo é instruído via *Prompt Engineering* a omitir cálculos espaciais (X, Y, Z) e exportar as conexões semânticas na forma de um grafo serializado em JSON. Essa matriz lógica reflete relações hierárquicas, ramificações e pré-requisitos mecânicos (portas trancadas vs. chaves).

2. Validação Determinística: O Godot intercepta o arquivo JSON e processa o grafo gerado através de um sistema baseado em regras clássicas. O algoritmo averígua condicionais estritas necessárias para uma fase funcional. As regras avaliam premissas topológicas, como: a existência imperativa de uma única sala inicial; a alcançabilidade da sala do chefe; a não alocação de chaves atrás da porta que elas abrem, tornando a sala inacessível; e a garantia de que as salas bônus (como tesouros) não estrangulem a rota principal de progresso. Caso o grafo apresente anomalias lógicas impeditivas, o sistema está sendo modelado para recusar o mapa e retroalimentar o LLM com as diretrizes do erro lógico.

3. Construção do Mapa e Testes Espaciais: Validado o grafo, o gerador espacial entra em vigor. Ele realiza a alocação procedural *tile-based* ou via *prefabs* de salas de combate, corredores e transições (XU; VERBRUGGE, 2025b). O algoritmo determinístico calcula dinamicamente as posições espaciais com base no tamanho individual de cada malha tridimensional, impedindo sobreposições visuais e inconsistência física (*clipping*). Subsequentemente, a *engine* aplica ferramentas nativas, como nós de *NavigationRegion3D* e projeção de *RayCasts*, para atestar que o caminho principal possui uma malha de navegação (NavMesh) intacta, conectando fisicamente a entrada à saída.

3 Metodologia Experimental e Resultados Esperados

Tendo em vista que o trabalho encontra-se na fase de desenvolvimento e implementação arquitetural, esta seção detalha a formatação dos experimentos programados para validar a premissa da pesquisa, bem como as inferências qualitativas e quantitativas previstas.

3.1 Delineamento dos Experimentos

Para atestar o impacto da validação determinística, os experimentos serão divididos em dois tratamentos comparativos gerando amostras de fases de dimensões variadas (pequena, média e grande escala, traduzidas em número de cômodos):

- **Grupo Controle (Geração Unificada):** O LLM será solicitado a gerar não somente as conexões lógicas, mas também as coordenadas 3D de instanciação para cada *asset* e cômodo.
- **Grupo Experimental (Geração Segregada):** A metodologia proposta na qual o LLM produz estritamente a árvore/grafo JSON da progressão, com o algoritmo determinístico encarregado de resolver e instanciar os obstáculos e coordenadas espaciais.

As amostras de fases resultantes de ambos os tratamentos serão expostas à verificações por meio de rotinas automatizadas no Godot que auditarão: (a) o número de colisões estruturais inválidas (sobreposição geométrica); (b) o tempo (milissegundos) para computar a malha navegável do início ao fim do nível; (c) a presença de bloqueios mecânicos (ex: chaves retidas em salas incomunicáveis).

3.2 Resultados Esperados

De acordo com o estado da arte e observações empíricas com modelos de linguagem aplicados a problemas de *Facility Layout Optimization* (XU; VERBRUGGE, 2025b), preveem-se os seguintes resultados primários:

- **Eliminação de Anomalias Geométricas:** Espera-se que a abordagem segregada demonstre uma taxa de falhas espaciais tendendo a zero, visto que a instaciação passará a obedecer a matrizes de colisão rigorosas que não são compreendidas eficientemente pela rede estocástica do LLM.
- **Resiliência e Coerência de *Design*:** Pela validação por grafo anterior à instanciação espacial, a metodologia deve impedir inteiramente a geração de bloqueios intransponíveis provocados por falhas semânticas em rotas ramificadas, permitindo níveis longos e coesos que combinem salas de recompensas com salas de desafio.
- **Automatização em *Addon*:** Como substrato prático da pesquisa, espera-se finalizar um *plugin* integrado à interface nativa do Godot. O pacote proverá um campo de *prompt*

diretamente no inspetor do editor, convertendo linguagem natural em cenários físicos editáveis, otimizando drasticamente o *workflow* do artista técnico ou *Level Designer*.

4 Considerações Finais

O presente trabalho propõe atacar a deficiência inerente dos LLMs no raciocínio espacial contínuo transferindo os cálculos matemáticos para as *game engines*. A arquitetura em desenvolvimento pauta-se no conceito de cooperação entre um componente estocástico-criativo (o modelo, como arquiteto) e um componente determinístico-rígido (o algoritmo, como engenheiro).

À medida em que a prova de conceito (*Addon* para Godot) avançar em testes empíricos de malhas navegáveis e consistência estrutural, será possível aferir a eventual redução da taxa de mapas inacessíveis, validando o arranjo teórico. Por fim, caso o método demonstre melhoras na solução problema investigado, projeta-se que a simbiose entre as descrições de *layout* via LLM e algoritmos de construtivismo direcional fomentará metodologias de prototipagem hiper-ágeis para o mercado de jogos digitais, permitindo aos desenvolvedores concentrarem-se primordialmente na parte artística, enquanto delegam o trabalho estrutural bruto a modelos inteligentes aliados à lógica formal.

5 Fundamentação Teórica

Este capítulo apresenta os fundamentos teóricos que sustentam o estudo da geração automática de fases tridimensionais para jogos digitais. A exposição organiza-se de modo progressivo: parte-se da definição de fases, de *Level Design* e da distinção conceitual entre organização lógica e organização espacial; em seguida, abordam-se os princípios da Geração Procedural de Conteúdo (PCG), sua vertente baseada em aprendizado de máquina (PCGML) e a caracterização dos Modelos de Linguagem de Grande Porte (LLMs), incluindo suas limitações de raciocínio e ancoragem espacial. Por fim, discutem-se a modelagem lógica de fases por meio da Teoria dos Grafos, a representação modular e geométrica de *layouts* tridimensionais, as arquiteturas híbridas guiadas por restrições e os critérios formais de validade, navegabilidade e solvabilidade.

5.1 Fases de Jogos, Level Design e Layouts 3D

5.1.1 Fases, Níveis e Level Design

No contexto dos jogos digitais, uma fase — também denominada nível (*level*) ou mapa — constitui a unidade espacial, temporal e interativa na qual as regras e mecânicas do sistema de regras do jogo se manifestam para o jogador (SMITH; WHITEHEAD; MATEAS, 2010; SHAKER; TOGELIUS; NELSON, 2016). Mais do que um cenário estático, uma fase opera como um contêiner estruturado para a experiência de jogo (*gameplay*), delimitando o território de exploração, os recursos disponíveis, os obstáculos a serem transpostos e as condições de vitória ou transição (SMITH; WHITEHEAD; MATEAS, 2010; LINDEN; LOPES; BIDARRA, 2014). Em gêneros como ação e aventura, jogos de plataforma e *Role-Playing Games* (RPGs), a qualidade da experiência interativa depende diretamente de como esses espaços são construídos e articulados (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2014).

A disciplina responsável pelo planejamento, concepção e estruturação de fases é o *Level Design* (DORMANS, 2010; SMITH; WHITEHEAD; MATEAS, 2010; TOTTEN, 2014). Seus objetivos fundamentais abrangem a condução da atenção e do deslocamento do jogador no espaço tridimensional, a introdução gradual de mecânicas, o balanceamento entre desafio e recompensa e a regulação do ritmo da experiência (*pacing*) (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2013, 2014; TOTTEN, 2014). Em fases do tipo masmorra (*dungeon*), por exemplo, o *Level Design* busca conciliar a liberdade de exploração espacial com um controle rigoroso sobre a progressão de desafios e quebra-cabeças (*puzzles*), assegurando que o jogador não encontre obstáculos intransponíveis para o conjunto de habilidades ou recursos que detém em determinado instante da partida (LINDEN; LOPES; BIDARRA, 2014; SHAKER; LIAPIS et al., 2016).

5.1.2 Layout de uma Fase

O *layout* de uma fase refere-se ao arranjo estrutural e espacial dos elementos que compõem o ambiente jogável (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2014). Em ambientes internos e masmorras, o *layout* é classicamente decomposto em unidades arquitetônicas discretas, destacando-se as salas (ou câmaras), os corredores (ou vias de transição), os conectores (portas, portais, escadas e elevadores) e as regiões funcionais (LINDEN; LOPES; BIDARRA, 2014; SHAKER; LIAPIS et al., 2016; XU; VERBRUGGE, 2025a). As salas atuam tipicamente como arenas de interação concentrada — abrigando combates, enigmas, recursos ou eventos narrativos —, ao passo que os corredores estabelecem as ligações físicas entre essas áreas, modulando o tempo de travessia e a visibilidade entre setores adjacentes (LINDEN; LOPES; BIDARRA, 2013; SHAKER; LIAPIS et al., 2016).

Além da geometria estrutural que delimita o espaço navegável (pisos, paredes, tetos e desníveis), o *layout* compreende a disposição interna de elementos funcionais e estéticos, frequentemente denominados instalações ou objetos de cena (*facilities* ou *props*), bem como entidades dinâmicas, tais como personagens não jogáveis (NPCs), inimigos, armadilhas e itens coletáveis (LINDEN; LOPES; BIDARRA, 2014; XU; VERBRUGGE, 2025b). Em ambientes tridimensionais (3D), a especificação de um *layout* envolve não apenas a distribuição planar em dois eixos, mas também a verticalidade, a sobreposição de múltiplos pavimentos, as relações de linha de visão (*line of sight*) e as dimensões volumétricas exatas de cada componente no espaço contínuo (XU; VERBRUGGE, 2025b,a).

5.1.3 Organização Lógica e Organização Espacial

Um princípio teórico central na literatura de *Level Design* consiste na distinção analítica entre duas estruturas complementares que coexistem em uma fase: a *missão* (organização lógica) e o *espaço* (organização espacial) (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2014). A missão corresponde à sequência lógica ou parcialmente ordenada de tarefas, ações e objetivos que o jogador deve cumprir para concluir o nível, tais como derrotar um guardião, obter uma chave específica, desativar um bloqueio e alcançar a saída (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2013). O espaço, por sua vez, refere-se à realização física e geométrica do ambiente virtual onde essas tarefas estão situadas, englobando as dimensões das salas, as distâncias métricas e a configuração arquitetônica das conexões (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2014).

Embora o espaço físico acomode a missão e esta seja mapeada sobre aquele, as duas estruturas são conceitualmente independentes e regidas por princípios distintos (DORMANS, 2010). Uma missão estritamente linear — na qual os objetivos precisam ser cumpridos em uma ordem única e fixa — pode ser distribuída sobre um espaço físico não linear, como um arranjo em estrela (*hub-and-spoke*) ou em anel, exigindo que o jogador retorne a áreas previamente visitadas à medida que desbloqueia novos acessos (DORMANS, 2010). Inversamente, uma missão não linear, dotada de múltiplos desafios

paralelos ou caminhos alternativos, pode ser acomodada em uma geometria espacial simples (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2013). Essa dissociação teórica evidencia que a coerência da progressão de um nível reside primariamente em sua estrutura lógica de dependências, enquanto sua viabilidade física e navegacional depende da correta realização geométrica no espaço tridimensional (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2013, 2014).

5.2 Geração Procedural de Conteúdo

5.2.1 Conceito e Objetivos da PCG

A Geração Procedural de Conteúdo (PCG — *Procedural Content Generation*) é definida na literatura como a criação algorítmica de conteúdos para jogos digitais de maneira autônoma ou com intervenção humana limitada e indireta (SHAKER; TOGELIUS; NELSON, 2016; SUMMERVILLE et al., 2018; MALEKI; ZHAO, 2024). O escopo do termo "conteúdo" abrange praticamente todos os artefatos que integram um jogo, incluindo níveis, mapas, terrenos, vegetação, texturas, efeitos sonoros, narrativas, missões (*quests*) e conjuntos de regras, distinguindo-se do comportamento em tempo de execução dos agentes de inteligência artificial que controlam NPCs (SHAKER; TOGELIUS; NELSON, 2016; SUMMERVILLE et al., 2018; MALEKI; ZHAO, 2024).

Historicamente introduzida em títulos pioneiros da década de 1980, como *Rogue* (1980) e *Elite* (1984), para contornar severas restrições de memória e armazenamento por meio da síntese algorítmica em tempo de execução (SHAKER; TOGELIUS; NELSON, 2016; SUMMERVILLE et al., 2018; ARAÚJO; ARANHA; MADEIRA, 2018), a PCG consolidou-se tanto na indústria quanto na pesquisa acadêmica em função de múltiplos objetivos (LINDEN; LOPES; BIDARRA, 2014; SHAKER; TOGELIUS; NELSON, 2016). Dentre as principais motivações para o seu emprego destacam-se: (i) a automação e a redução do tempo e do custo financeiro associados à autoria manual de ativos em larga escala; (ii) o aumento da variabilidade e da diversidade dos ambientes gerados, ampliando o fator de rejogabilidade (*replayability*); (iii) o suporte à adaptabilidade centrada no jogador, permitindo ajustar dinamicamente os desafios às habilidades observadas; e (iv) a exploração de novas estéticas e mecânicas emergentes (LINDEN; LOPES; BIDARRA, 2014; SHAKER; TOGELIUS; NELSON, 2016; SUMMERVILLE et al., 2018).

A avaliação e o projeto de geradores procedurais apoiam-se em propriedades técnicas fundamentais, notadamente a reprodutibilidade, a expressividade e a controlabilidade (SMITH; WHITEHEAD; MATEAS, 2010; SMITH; WHITEHEAD, 2010; LINDEN; LOPES; BIDARRA, 2014; MALEKI; ZHAO, 2024). A reprodutibilidade decorre frequentemente do uso de Geradores de Números Pseudoaleatórios (PRNGs), nos quais a especificação de um valor inicial — denominado semente (*seed*) — permite reconstruir de maneira determinística a mesma sequência de decisões algorítmicas (MALEKI; ZHAO, 2024). A expressividade, caracterizada pela análise do espaço expressivo (*expressive range*), quantifica e mapeia a variedade, a distribuição e as propriedades estilísticas ou

estruturais dos artefatos que o gerador é capaz de produzir no espaço de design (SMITH; WHITEHEAD, 2010; LINDEN; LOPES; BIDARRA, 2013). A controlabilidade, por sua vez, reflete a capacidade do projetista ou do sistema de direcionar intencionalmente as características do conteúdo produzido por meio de parâmetros, regras ou restrições (LINDEN; LOPES; BIDARRA, 2014; SHAKER; LIAPIS et al., 2016). Um desafio central no projeto de geradores consiste em equilibrar a tensão entre controle e diversidade: restrições excessivamente rígidas podem limitar a variabilidade dos resultados, ao passo que a ausência de controle eleva o risco de geração de artefatos inválidos ou incoerentes (SMITH; WHITEHEAD, 2010; LINDEN; LOPES; BIDARRA, 2014).

5.2.2 Principais Abordagens de PCG

Uma classificação recorrente na literatura de PCG organiza os métodos generativos em três grandes paradigmas operacionais: métodos construtivos, métodos baseados em busca (*search-based*) e métodos baseados em restrições ou solucionadores (*solver-based* / *constraint-based*) (TOGELIUS et al., 2011; SHAKER; TOGELIUS; NELSON, 2016; SUMMERVILLE et al., 2018; MALEKI; ZHAO, 2024).

Os métodos construtivos caracterizam-se por gerar o conteúdo em uma única passagem sequencial de passos algorítmicos, com tempo de execução previsível e sem realizar ciclos iterativos de avaliação e regeneração sobre o artefato produzido (SHAKER; LIAPIS et al., 2016; SUMMERVILLE et al., 2018). Exemplos clássicos incluem funções de ruído coerente (como o ruído de Perlin para síntese de terrenos e texturas), autômatos celulares, algoritmos de particionamento espacial e gramáticas generativas (LINDEN; LOPES; BIDARRA, 2014; SHAKER; LIAPIS et al., 2016; MALEKI; ZHAO, 2024). Importa ressaltar que procedimentos construtivos podem fornecer garantias estruturais formais sobre o conteúdo gerado (como ausência de sobreposição ou conectividade garantida) quando suas regras forem rigorosamente formuladas para preservar tais invariantes em cada etapa da construção (SHAKER; LIAPIS et al., 2016). Por outro lado, caso dependam de decisões puramente estocásticas desprovidas de verificação antecipada, o determinismo algorítmico assegura apenas a repetibilidade da saída a partir de uma semente fixa, não prevenindo falhas estruturais ou becos sem saída (LINDEN; LOPES; BIDARRA, 2014; SHAKER; LIAPIS et al., 2016).

Os métodos baseados em busca (*search-based PCG*) enquadram-se como um caso particular do paradigma gerar-e-testar (*generate-and-test*) guiado por otimização (TOGELIUS et al., 2011; SHAKER; TOGELIUS; NELSON, 2016; MALEKI; ZHAO, 2024). Em vez de aceitar imediatamente o artefato sintetizado ou simplesmente descartá-lo de forma binária, algoritmos baseados em busca atribuem a cada instância candidata um valor numérico contínuo por meio de uma função de avaliação ou aptidão (*fitness function*), utilizando esse escore para guiar iterativamente a modificação e a seleção de novas soluções no espaço de busca (TOGELIUS et al., 2011; LINDEN; LOPES; BIDARRA, 2014; ARAÚJO; ARANHA; MADEIRA, 2018; MALEKI; ZHAO, 2024). Essa categoria

emprega amplamente técnicas de computação evolutiva (como Algoritmos Genéticos) (TOGELIUS et al., 2011; LINDEN; LOPES; BIDARRA, 2014; MALEKI; ZHAO, 2024), além de meta-heurísticas como têmpera simulada (*Simulated Annealing*), otimização por enxame de partículas (PSO) e Busca em Árvore de Monte Carlo (MCTS) (MALEKI; ZHAO, 2024; XU; VERBRUGGE, 2025b).

Os métodos baseados em restrições (*constraint-based* ou *solver-based PCG*), por sua vez, especificam declarativamente as propriedades e relações obrigatórias que o artefato final deve satisfazer, delegando a construção a solucionadores matemáticos ou lógicos (*constraint solvers*) (SMITH; WHITEHEAD; MATEAS, 2010; SMITH; MATEAS, 2011; LINDEN; LOPES; BIDARRA, 2014; SHAKER; TOGELIUS; NELSON, 2016; SUMMERVILLE et al., 2018). Destacam-se nesse paradigma a Programação por Conjuntos de Respostas (*Answer Set Programming* — ASP), que formaliza regras e restrições de design como predicados lógicos satisfatíveis (SMITH; MATEAS, 2011), solucionadores de satisfatibilidade módulo teorias (SMT) e solucionadores de Problemas de Satisfação de Restrições (CSP) (SMITH; WHITEHEAD; MATEAS, 2010; CHEN; JHALA, 2025). Integram também essa vertente os algoritmos baseados em propagação local de restrições de adjacência inspirados em mecânica estatística, com destaque para o *WaveFunctionCollapse* (WFC) (KARTH; SMITH, 2017; MALEKI; ZHAO, 2024). Nessa abordagem, o foco desloca-se da definição de passos procedurais imperativos para a formulação matemática e lógica do espaço de design admissível (SMITH; WHITEHEAD; MATEAS, 2010; SMITH; MATEAS, 2011; LINDEN; LOPES; BIDARRA, 2014).

5.2.3 Geração Procedural de Fases

A aplicação de PCG à criação de fases e níveis constitui um dos tópicos mais investigados na área, apresentando exigências técnicas particulares que a diferenciam da geração de ativos puramente cosméticos (SUMMERVILLE et al., 2018; MALEKI; ZHAO, 2024). A literatura distingue conteúdos cosméticos (como texturas e efeitos visuais, cuja alteração não modifica o resultado das ações do jogador) de conteúdos funcionais, que afetam diretamente a mecânica e a dinâmica da partida (SUMMERVILLE et al., 2018). Fases de jogos são artefatos intrinsecamente funcionais: uma fase visualmente atraente, mas que apresente falhas topológicas ou bloqueios estruturais que impeçam sua conclusão, é considerada inválida para o propósito do jogo (SUMMERVILLE et al., 2018; LINDEN; LOPES; BIDARRA, 2014).

Na geração procedural de níveis e masmorras, um método típico envolve três componentes conceituais: (1) um modelo representacional abstrato que descreve a estrutura global da fase; (2) um método para construir esse modelo representacional; e (3) um método para sintetizar a geometria efetiva da fase a partir do modelo abstrato (LINDEN; LOPES; BIDARRA, 2014; SHAKER; LIAPIS et al., 2016). Diversas famílias algorítmicas foram desenvolvidas para essas etapas. Algoritmos de particionamento de espaço, como o Particionamento Binário do Espaço (*Binary Space Partitioning* — BSP), *quadtrees* (em

2D) e *octrees* (em 3D), subdividem recursivamente o domínio espacial em células disjuntas, garantindo estruturalmente que salas alocadas em partições distintas não se sobreponham (SHAKER; LIAPIS et al., 2016). Em contrapartida, abordagens baseadas em agentes escavadores (*agent-based dungeon growing*) operam em escala local, deslocando um agente virtual que escava túneis e posiciona salas de maneira passo a passo; embora produzam configurações orgânicas, agentes sem mecanismos de inspeção antecipada (*look-ahead*) estão sujeitos a gerar salas sobrepostas e corredores sem saída (SHAKER; LIAPIS et al., 2016).

De modo complementar, autômatos celulares operam sobre grades discretas aplicando regras de transição baseadas na vizinhança local (como as vizinhanças de Moore ou de von Neumann) para formar cavernas naturais, mas não asseguram, por si sós, a conectividade global entre câmaras isoladas, exigindo etapas adicionais de verificação e abertura de passagens (LINDEN; LOPES; BIDARRA, 2014; SHAKER; LIAPIS et al., 2016). Gramáticas generativas — incluindo gramáticas de grafos e gramáticas de formas (*shape grammars*) — expandem um axioma inicial por meio de regras de produção e reescrita, permitindo capturar princípios de progressão e hierarquia em níveis de ação e aventura (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2013; SHAKER; LIAPIS et al., 2016). Apesar da maturidade dessas técnicas em domínios bidimensionais, a transição para a geração de fases tridimensionais complexas permanece um desafio aberto na literatura devido ao aumento expressivo das restrições geométricas, de navegação vertical e de acoplamento espacial (LINDEN; LOPES; BIDARRA, 2014; MALEKI; ZHAO, 2024; XU; VERBRUGGE, 2025b).

5.3 PCG Baseada em Aprendizado de Máquina e Uso de LLMs

5.3.1 Procedural Content Generation via Machine Learning — PCGML

A Geração Procedural de Conteúdo via Aprendizado de Máquina (*Procedural Content Generation via Machine Learning* — PCGML) é definida formalmente como a geração de conteúdos de jogos diretamente por modelos estatísticos ou de aprendizado de máquina treinados sobre conjuntos de conteúdos preexistentes de jogos (SUMMERVILLE et al., 2018). A distinção fundamental entre o PCGML e as abordagens clássicas baseadas em busca ou em solucionadores reside no mecanismo gerador: enquanto a busca evolucionária pode eventualmente empregar um modelo neural apenas como função de avaliação para pontuar candidatos sintetizados por outros meios, no PCGML o modelo treinado internaliza a distribuição estatística do *corpus* de entrada e amostra diretamente novas instâncias a partir de sua inferência (SUMMERVILLE et al., 2018).

A taxonomia clássica de PCGML organiza os métodos segundo a representação subjacente dos dados (sequências unidimensionais, grades bidimensionais/tridimensionais e grafos) e o paradigma de modelagem empregado (SUMMERVILLE et al., 2018). Entre as técnicas consolidadas figuram: (i) modelos probabilísticos de contagem de frequência, como modelos de n-gramas, Cadeias de Markov Multidimensionais (MdMCs) e Campos

Aleatórios de Markov (MRFs); (ii) métodos de redução de dimensionalidade e fatoração de matrizes (como PCA); (iii) Redes Neurais Recorrentes, especialmente redes *Long Short-Term Memory* (LSTMs), utilizadas na geração sequencial de fases baseadas em blocos; (iv) *Autoencoders* e *Variational Autoencoders* (VAEs), que mapeiam fases em espaços latentes contínuos para fins de interpolação e reparo de segmentos; e (v) Redes Adversárias Generativas (GANs) (SUMMERVILLE et al., 2018; MALEKI; ZHAO, 2024).

No âmbito do aprendizado por reforço aplicado à geração (PCGRL), o paradigma difere do treinamento supervisionado sobre *corpora* estáticos (KHALIFA et al., 2020): o agente gerador interage dinamicamente com um ambiente de desenvolvimento de fases, modificando células ou inserindo peças por meio de ações discretas e sendo otimizado para maximizar uma função de recompensa cumulativa que quantifica propriedades de jogabilidade e solvabilidade (KHALIFA et al., 2020; MALEKI; ZHAO, 2024).

É conceitualmente relevante distinguir a utilização de Modelos de Linguagem de Grande Porte (LLMs) em relação à definição estrita de PCGML formulada por Summerville et al. (SUMMERVILLE et al., 2018). Quando um LLM fundacional é empregado por meio de engenharia de *prompt* em regime de zero-shot ou few-shot, o modelo não foi treinado sobre um *corpus* de fases de jogos específico, mas opera por meio de aprendizado em contexto (*in-context learning*) suportado pelo conhecimento estrutural e semântico adquirido durante o pré-treinamento geral em linguagem natural e código (BROWN et al., 2020; GALLOTTA et al., 2024). Por outro lado, caso um modelo de linguagem seja expressamente submetido a ajuste fino (*fine-tuning*) sobre conjuntos de dados de fases (como representações em texto, ASCII ou grafos de níveis), sua operação alinha-se estritamente à definição clássica de PCGML (SUMMERVILLE et al., 2018; TODD et al., 2023).

Embora o aprendizado a partir de exemplos atenue a necessidade de codificação manual de regras estilísticas, a literatura aponta duas limitações estruturais no paradigma de PCG baseado em modelos conexionistas (SUMMERVILLE et al., 2018; MALEKI; ZHAO, 2024; HER et al., 2025): a escassez de dados (*data scarcity*), decorrente do número restrito de fases digitalizadas e da ausência de representações universais entre diferentes jogos (SUMMERVILLE et al., 2018); e a dificuldade de garantir propriedades funcionais globais, visto que modelos puramente estatísticos tendem a reproduzir padrões visuais plausíveis localmente, mas não oferecem garantias determinísticas de conectividade, acessibilidade ou solvabilidade global sem módulos externos de validação (SUMMERVILLE et al., 2018; MALEKI; ZHAO, 2024).

5.3.2 Modelos de Linguagem de Grande Porte Aplicados à Geração de Conteúdo

Os Modelos de Linguagem de Grande Porte (*Large Language Models* — LLMs) constituem uma classe de modelos fundacionais de aprendizado profundo fundamentados na arquitetura *Transformer* (VASWANI et al., 2017; GALLOTTA et al., 2024). Historicamente, os modelos neurais sequenciais precursores em Processamento de Linguagem

Natural (PLN), tais como Redes Neurais Recorrentes (RNNs) e LSTMs, enfrentavam severos entraves para capturar dependências de longo alcance devido ao fenômeno do esvanecimento ou explosão de gradientes (*vanishing/exploding gradients*) durante o treinamento por retropropagação ao longo do tempo (BENGIO; SIMARD; FRASCONI, 1994). A arquitetura *Transformer* superou essa barreira teórica ao prescindir da recorrência sequencial e introduzir o mecanismo de autoatenção (*self-attention*) (VASWANI et al., 2017). O mecanismo de autoatenção calcula diretamente coeficientes de ponderação entre todos os pares de elementos (*tokens*) da sequência de entrada, viabilizando o processamento altamente paralelizado e permitindo correlacionar informações estruturais e semânticas independentemente de sua distância posicional no texto (VASWANI et al., 2017; GALLOTTA et al., 2024).

Na vertente generativa preponderante, os modelos autorregressivos causais operam estimando a distribuição de probabilidade condicional do próximo *token* x_t com base no histórico de *tokens* precedentes:

$$P(x_1, x_2, \ldots, x_T) = \prod_{t=1}^{T} P(x_t \mid x_1, x_2, \ldots, x_{t-1}) \tag{1}$$

O ciclo de desenvolvimento desses modelos envolve tipicamente o pré-treinamento autossupervisionado em larga escala sobre extensos *corpora* de textos e linguagens de programação, seguido por ajuste fino supervisionado para seguimento de instruções (*instruction tuning*) e alinhamento por preferências (como RLHF) (BROWN et al., 2020; GALLOTTA et al., 2024; HER et al., 2025). Em modelos com bilhões de parâmetros, destaca-se a capacidade de aprendizado em contexto (*In-Context Learning* — ICL), que possibilita a resolução de novas tarefas a partir exclusivamente de especificações no *prompt* em regime *zero-shot* ou condicionada por poucos exemplos demonstrativos (*few-shot*), sem modificação dos pesos sinápticos da rede (BROWN et al., 2020). Estruturações avançadas de *prompting*, como Cadeia de Pensamento (*Chain-of-Thought* — CoT) e a incorporação de Geração Aumentada por Recuperação (*Retrieval-Augmented Generation* — RAG), estendem essa capacidade permitindo decomposição analítica de etapas e consulta a fontes de dados estruturados (GALLOTTA et al., 2024; TANG et al., 2025; WANG et al., 2026).

Na literatura de PCG, o emprego de LLMs diversificou-se em múltiplas funções especializadas (GALLOTTA et al., 2024; MALEKI; ZHAO, 2024):

1. **Gerador direto de representações e conteúdo:** síntese de artefatos textuais como diálogos e narrativas, bem como mapas bidimensionais codificados diretamente em texto ou caracteres ASCII (como grades para *Sokoban* ou *Super Mario Bros.*) (TODD et al., 2023; NASIR; JAMES; TOGELIUS, 2024);

2. **Planejador lógico e semântico de alto nível:** geração de estruturas conceituais abstratas, incluindo árvores de missões, grafos de progressão lógica e atribuição contextual de temas narrativos a salas e regiões (GALLOTTA et al., 2024; XU; VERBRUGGE, 2025a; WANG et al., 2026);

3. **Sintetizador de parâmetros e código para ferramentas procedurais:** produção de rotinas em linguagens de programação (como scripts para motores ou modeladores 3D), parametrização de geradores construtivos e formulação de restrições para solucionadores matemáticos (HU et al., 2024; TANG et al., 2025; XU; VERBRUGGE, 2025b; YANG et al., 2026);

4. **Assistente de design em sistemas de iniciativa mista:** atuação colaborativa na interpretação de comandos do designer, oferecendo sugestões estruturais e refinando especificações conceituais (GALLOTTA et al., 2024; BANDEALI, 2026; WANG et al., 2026).

5.3.3 Geração de Representações Estruturadas

Quando modelos de linguagem são integrados a sistemas computacionais externos — tais como motores gráficos, otimizadores matemáticos ou verificadores formais —, a emissão de respostas em linguagem natural livre torna-se inadequada devido à sua ambiguidade intrínseca e à suscetibilidade a falhas sintáticas na análise automatizada (*parsing*) (XU; VERBRUGGE, 2025b; REGMI, 2025). Para garantir a interoperabilidade entre a inferência probabilística do LLM e os procedimentos algorítmicos determinísticos, a literatura preconiza a geração de representações estruturadas intermediárias (HU et al., 2024; TANG et al., 2025; XU; VERBRUGGE, 2025b; CHEN; JHALA, 2025; WANG et al., 2026).

Sob o ponto de vista conceitual e de engenharia de software, é indispensável diferenciar o *modelo conceitual* do *formato de serialização* (XU; VERBRUGGE, 2025b,a; CHEN; JHALA, 2025):

- O **modelo conceitual** (ou estrutura abstrata de informação) é a formalização lógica ou matemática que organiza o domínio do problema. Exemplos compreendem grafos de cena e de progressão (XU; VERBRUGGE, 2025a; WANG et al., 2026), conjuntos de triplas relacionais do tipo `[Objeto-Relação-Objeto]` (CHEN; JHALA, 2025), hierarquias de componentes de cena, linguagens de descrição de domínio específico (DSL) como a GLDL (*Game Level Description Language*) (XU; VERBRUGGE, 2025b), ou sínteses procedurais via código executável (HU et al., 2024; YANG et al., 2026);

- O **formato de serialização** é o padrão textual ou binário adotado para codificar, armazenar e transmitir esse modelo conceitual entre diferentes subsistemas (SUN et al., 2025; XU; VERBRUGGE, 2025a). O formato JSON (*JavaScript Object Notation*), bem como esquemas em XML ou YAML, constitui a representação hegemônica por estruturar dados hierárquicos em pares chave-valor e arranjos vetoriais com suporte a validação rigorosa de sintaxe por meio de esquemas formais (*JSON Schema*) (TANG et al., 2025; SUN et al., 2025; XU; VERBRUGGE, 2025a). Abordagens pontuais na literatura, a exemplo de Öcal et al. (ÖCAL et al., 2024), adaptaram

experimentalmente a sintaxe de folhas de estilo (CSS) para declarar parâmetros de caixas delimitadoras; não obstante, o padrão JSON consolida-se como a convenção primária de serialização para intercâmbio de dados estruturados.

O formato de serialização não se confunde, portanto, com a lógica do nível: ele é o meio físico-textual de transmissão que transporta a abstração formal da fase para que ferramentas computacionais possam interpretá-la e validá-la de modo determinístico (TANG et al., 2025; XU; VERBRUGGE, 2025a).

5.4 Raciocínio e Ancoragem Espacial em LLMs

5.4.1 Raciocínio Espacial

O raciocínio espacial compreende a capacidade de representar formalmente entidades em um domínio multidimensional, inferir suas propriedades geométricas e deduzir os desdobramentos de suas transformações e arranjos mútuos (COHN; RENZ, 2008; SUN et al., 2025; XU; VERBRUGGE, 2025b). No domínio do *Level Design* e da síntese de ambientes virtuais 3D, o raciocínio espacial subdivide-se entre relações qualitativas (topológicas e mereológicas) (COHN; RENZ, 2008) e relações quantitativas métricas contínuas (XU; VERBRUGGE, 2025b; CHEN; JHALA, 2025).

Na formulação da organização espacial de uma fase, identificam-se propriedades e relações geométricas essenciais (COHN; RENZ, 2008; SUN et al., 2025; XU; VERBRUGGE, 2025b; CHEN; JHALA, 2025):

- **Posição:** localização absoluta ou relativa de uma entidade no espaço $\mathbb{R}^3$, expressa em relação à origem global ou a um ponto de referência local;
- **Distância e proximidade:** separação métrica euclidiana entre entidades, determinando adjacência funcional, raio de alcance ou zonas de segurança;
- **Direção versus Orientação:** sob o rigor do raciocínio espacial (COHN; RENZ, 2008), a *direção* constitui uma relação espacial relativa entre duas entidades (indicando, por exemplo, que a sala B está situada ao norte, a leste ou acima da sala A), enquanto a *orientação* refere-se ao alinhamento angular intrínseco de uma entidade (sua *pose* ou vetor de mira frontal — *facing* — no espaço, determinado por suas rotações em torno dos eixos locais);
- **Adjacência e conectividade:** contato geométrico ou comunicação de passagem entre regiões contíguas, tais como salas que compartilham portais comuns;
- **Contenção:** exigência topológica e volumétrica de que determinado conjunto de elementos (como itens ou mobílias) esteja integralmente situado dentro do perímetro delimitador de uma sala ou compartimento hospedeiro;
- **Interseção e ocupação volumétrica:** verificação de compartilhamento de pontos no espaço tridimensional entre dois ou mais corpos físicos.

A preservação concorrente da consistência de todas essas relações ao longo da alocação de dezenas de elementos arquitetônicos e funcionais constitui um problema combinatório e contínuo de alta complexidade (XU; VERBRUGGE, 2025b; FENG et al., 2026).

5.4.2 Spatial Grounding

O problema de ancoragem semântica (*symbol grounding problem*), formulado originalmente por Harnad (HARNAD, 1990), investiga como símbolos abstratos manipulados puramente com base em regras sintáticas podem adquirir significado intrínseco e correspondência funcional com entidades concretas de um mundo físico ou simulado. No âmbito dos sistemas generativos aplicados a ambientes 3D, a *ancoragem espacial* (*spatial grounding*) designa a capacidade de mapear comandos linguísticos e especificações conceituais (como "posicionar uma mesa no centro da câmara" ou "vincular o corredor de acesso à ala norte") em coordenadas métricas coerentes, dimensões tridimensionais compatíveis e restrições espaciais verificáveis (HARNAD, 1990; HU et al., 2024; SUN et al., 2025; CHEN; JHALA, 2025).

Um componente crítico da ancoragem espacial consiste na interpretação correta de *affordances*. Na definição clássica da psicologia ecológica de Gibson (GIBSON, 1979), uma *affordance* traduz as possibilidades reais de ação que os elementos do ambiente físico oferecem a um agente (por exemplo, uma porta aberta oferece passagem, uma superfície horizontal firme oferece suporte para caminhar ou sentar, e uma parede oferece bloqueio ou proteção) (GIBSON, 1979). A ancoragem espacial requer que o sistema não apenas reconheça a coocorrência semântica entre termos linguísticos, mas garanta que as configurações geométricas resultantes respeitem as propriedades funcionais de uso: corredores devem manter altura e largura livres para circulação de agentes, portas devem possuir desobstrução frontal para acesso e objetos de apoio devem respeitar o contato estável de superfícies no espaço tridimensional (SUN et al., 2025; XU; VERBRUGGE, 2025b).

5.4.3 Limitações dos LLMs em Tarefas Espaciais Tridimensionais

No contexto dos modelos generativos de linguagem, define-se por alucinação a síntese de respostas que apresentam correção superficial ou gramatical, mas que violam a coerência factual, contêm premissas contraditórias ou desrespeitam restrições explícitas do domínio (GALLOTTA et al., 2024; XU; VERBRUGGE, 2025b; BANDEALI, 2026). Quando os LLMs são incumbidos de determinar diretamente atributos geométricos contínuos em ambientes 3D — tais como coordenadas cartesianas (x, y, z), caixas delimitadoras e ângulos de rotação para múltiplos elementos interconectados —, esse comportamento manifesta-se sob a forma de *alucinação espacial* ou inconsistência métrica (GALLOTTA et al., 2024; SUN et al., 2025; YANG et al., 2026; FENG et al., 2026).

A fundamentação teórica dessa limitação decorre de características constitutivas dos modelos de linguagem (GALLOTTA et al., 2024; HU et al., 2024; SUN et al., 2025; FENG et al., 2026):

1. **Ausência de modelagem física e geométrica na inferência:** LLMs são treinados com base na verossimilhança estatística de sequências de *tokens* textuais, sem acesso nativo a um motor de cálculo vetorial ou representação interna de espaço euclidiano tridimensional durante o cálculo de autoatenção (GALLOTTA et al., 2024; SUN et al., 2025);

2. **Severa compressão de informação geométrica:** ao representar geometrias poligonais complexas exclusivamente por rótulos simbólicos e tuplas numéricas discretizadas, omitem-se as interações de superfície contínuas e as relações de contato entre sólidos (FENG et al., 2026);

3. **Deriva autorregressiva e acúmulo de erros métricos:** em processos de geração sequencial ou em planejamentos em passo único (*single-step*), pequenos desvios numéricos cometidos nas primeiras coordenadas preditas propagam-se recursivamente pelo contexto, degradando a consistência geométrica global conforme o número de elementos cresce no ambiente (SUN et al., 2025; FENG et al., 2026; YANG et al., 2026).

Diversos estudos empíricos recentes ilustram concretamente essas falhas:

- No sistema *SceneCraft*, Hu et al. (HU et al., 2024) evidenciaram que, ao gerar scripts em Python para modeladores 3D como o Blender, os LLMs conseguem estruturar a hierarquia de objetos, mas cometem graves imprecisões nas coordenadas numéricas, gerando colisões e desalinhamentos severos que exigem otimizadores numéricos acoplados para corrigir a cena;

- No framework *RoomPlanner*, Sun et al. (SUN et al., 2025) constataram empiricamente sobre o conjunto de dados Matterport3D que a solicitação direta de caixas delimitadoras a LLMs resulta em taxas substanciais de interpenetração de móveis e posicionamento de elementos fora das paredes dos cômodos;

- No estudo *Code-as-Room*, Yang et al. (YANG et al., 2026) reportaram que a síntese de recintos 3D via código por LLMs padece de alucinações espaciais e violações funcionais de circulação quando desprovida de verificação externa de navegabilidade.

As falhas observadas agrupam-se em quatro categorias recorrentes (SUN et al., 2025; XU; VERBRUGGE, 2025b; FENG et al., 2026; YANG et al., 2026): (i) **colisões e sobreposições volumétricas (*overlapping* / *clipping*)**, nas quais malhas sólidas ocupam o mesmo volume simultaneamente; (ii) **violações de limites e suporte (*out-of-bounds* e objetos flutuantes)**, em que elementos atravessam paredes externas ou ficam suspensos sem contato com o piso; (iii) **erros de rotação e alinhamento**, como portas e

armários posicionados de frente para paredes cegas; e (iv) **obstruções de circulação**, nas quais objetos bloqueiam corredores vitais e fragmentam o espaço navegável (SUN et al., 2025; XU; VERBRUGGE, 2025b,a).

Tais constatações consolidam a compreensão de que, embora os LLMs demonstrem notável competência para o raciocínio semântico e planejamento conceitual, a determinação direta de layouts tridimensionais estritamente por inferência de linguagem é inerentemente falha, demandando arquiteturas estruturadas que deleguem a instanciação geométrica e a validação espacial a componentes algorítmicos dedicados (HU et al., 2024; SUN et al., 2025; XU; VERBRUGGE, 2025b,a; YANG et al., 2026).

5.5 Representação Lógica de Fases por Grafos

5.5.1 Conceitos Fundamentais de Teoria dos Grafos

A Teoria dos Grafos fornece a base matemática formal para modelar relações discretas e conectividades estruturais em sistemas complexos (CORMEN et al., 2009; SUMMERVILLE et al., 2018). Formalmente, um grafo é definido como um par ordenado $G = (V, E)$, no qual V consiste em um conjunto finito e não vazio de vértices (ou nós) e E representa um conjunto de arestas (ou arcos) que conectam pares de vértices pertencentes a V (CORMEN et al., 2009; MALEKI; ZHAO, 2024).

Quanto à orientação de suas arestas, um grafo pode ser não direcionado — em que as arestas indicam conexões simétricas bidirecionais entre pares de vértices $\{u, v\}$ —, ou direcionado (dígrafo), quando cada aresta consiste em um par ordenado $(u, v) \in E$ orientado do vértice de origem u para o vértice de destino v (CORMEN et al., 2009; DORMANS, 2010). Em dígrafos, distingue-se o grau de entrada ($d^-(v)$, número de arestas que incidem em v) do grau de saída ($d^+(v)$, número de arestas que partem de v) (CORMEN et al., 2009). Um grafo ponderado associa valores numéricos reais às suas arestas (como distâncias métricas, tempos de travessia ou dificuldades), enquanto um grafo rotulado ou de atributos vincula tipos, símbolos e dicionários de propriedades tanto a vértices quanto a arestas (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2013, 2014; CHEN; JHALA, 2025).

Sobre essas estruturas, fundamentam-se propriedades topológicas cruciais (CORMEN et al., 2009; LINDEN; LOPES; BIDARRA, 2014; SUMMERVILLE et al., 2018):

- **Caminho (*path*):** sequência de vértices $\langle v_0, v_1, \ldots, v_k \rangle$ tal que $(v_{i-1}, v_i) \in E$ para todo $i \in \{1, \ldots, k\}$;
- **Ciclo:** caminho fechado no qual $v_0 = v_k$ e que contém pelo menos uma aresta distinta. Um grafo que não possui ciclos é dito acíclico;
- **Grafo Acíclico Direcionado (DAG — *Directed Acyclic Graph*):** dígrafo desprovido de caminhos orientados fechados, estrutura fundamental para expressar relações de precedência estrita e causalidade (CORMEN et al., 2009; WANG et al., 2026);

- **Conectividade e Conectividade Forte:** em grafos não direcionados, a conectividade assegura que exista ao menos um caminho entre qualquer par de vértices. Em dígrafos, define-se conectividade forte quando, para todo par de vértices $u, v \in V$, existem simultaneamente caminhos direcionados de u para v e de v para u (CORMEN et al., 2009);
- **Alcançabilidade (*reachability*):** condição na qual existe um caminho orientado viável partindo de um nó inicial v_{start} até um nó de destino v_{target} (CORMEN et al., 2009; XU; VERBRUGGE, 2025a);
- **Planaridade:** propriedade geométrica de um grafo que permite desenhá-lo em um plano bidimensional $\mathbb{R}^2$ de modo que suas arestas se interceptem unicamente nos vértices aos quais estão conectadas (CORMEN et al., 2009; LINDEN; LOPES; BIDARRA, 2013).

5.5.2 Grafos Aplicados ao Level Design

Na modelagem de fases e masmorras procedurais, a literatura apoia-se em grafos sob duas perspectivas complementares: grafos de espaço (topológicos) e grafos de missão (ou de ações) (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2013, 2014; SUMMERVILLE et al., 2018).

Nos *grafos topológicos de espaço*, cada vértice $v \in V$ representa uma região física discreta do nível (como uma sala, corredor, câmara ou pátio), e cada aresta $e \in E$ representa uma passagem física ou conexão de adjacência (portas, arcos de passagem, escadarias ou pontes) (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2014; SUMMERVILLE et al., 2018). Os vértices recebem atributos temáticos e funcionais (sala de início, sala de combate, câmara de tesouro, sala de chefe), enquanto as arestas recebem atributos de passagem (passagem aberta, porta trancada, passagem unidirecional com queda) (LINDEN; LOPES; BIDARRA, 2014; XU; VERBRUGGE, 2025a).

Nos *grafos de missão* ou *grafos de ações* (*action graphs*), por outro lado, os vértices representam eventos, tarefas ou ações que o jogador executa (ex.: obter chave, ativar alavanca, derrotar sentinela, transpor portal), e as arestas direcionadas codificam restrições causais de precedência no tempo (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2013). Conforme demonstrado seminalmente por Dormans (DORMANS, 2010) e aprofundado por van der Linden et al. (LINDEN; LOPES; BIDARRA, 2013, 2014), um grafo de ações pode ser mapeado em um grafo de salas mediante o agrupamento de ações que compartilham a mesma localização física (*co-located actions*), unificando a estrutura de progressão e a topologia arquitetônica.

5.5.3 Grafos de Progressão e Dependências

A integração formal entre a conectividade espacial e as mecânicas de jogo estabelece os *grafos de progressão*, nos quais arestas e vértices são condicionados a regras de acesso

lógico (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2013, 2014; XU; VERBRUGGE, 2025a). O paradigma central para estruturar a progressão em masmorras e fases de aventura é o quebra-cabeça de chave e fechadura (*lock-and-key puzzle*) (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2014; XU; VERBRUGGE, 2025a). Uma fechadura denota qualquer barreira ou pré-requisito funcional (uma porta física trancada, uma barreira mágica, um obstáculo ambiental intransponível), enquanto a chave corresponde ao elemento ou estado necessário para desativá-la (uma chave física, uma habilidade de salto adquirida, o acionamento de um botão remoto ou a eliminação de inimigos) (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2013; SHAKER; LIAPIS et al., 2016).

Do ponto de vista formal, a estrutura de dependências lógicas induz um subgrafo direcionado de pré-condições $G_{\text{dep}} = (V_{\text{dep}}, E_{\text{dep}})$, o qual deve satisfazer estritamente a condição de ser um Grafo Acíclico Direcionado (DAG) para garantir a inexistência de impasses circulares insolúveis (*deadlocks*) (CORMEN et al., 2009; XU; VERBRUGGE, 2025a). Em tal DAG de dependências, existe ao menos uma ordenação topológica $\tau : V_{\text{dep}} \to \{1, 2, \ldots, |V_{\text{dep}}|\}$ tal que, para qualquer aresta orientada $(u, v) \in E_{\text{dep}}$, tem-se $\tau(u) < \tau(v)$ (CORMEN et al., 2009). Isso significa que todo recurso ou chave u indispensável para a superação do bloqueio em v deve ser alcançável e coletado antes que o estado de travessia de v seja solicitado (XU; VERBRUGGE, 2025a).

É essencial pontuar o contraste estrutural entre a lógica de dependências e a circulação física: enquanto o grafo causal de pré-condições (G_{dep}) deve ser estritamente acíclico para ser solucionável, o grafo espacial de navegação do nível pode conter ciclos controlados (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2014). Esses ciclos na geometria física enriquecem a exploração ao fornecer rotas alternativas, atalhos destraváveis e a necessidade intencional de retorno a áreas previamente visitadas (*backtracking*) após a obtenção de novos recursos (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2013, 2014). A solvabilidade do nível modela-se formalmente no espaço de estados do jogo (S), no qual cada estado $s \in S$ é composto pela localização espacial do agente associada ao seu inventário corrente de chaves, itens e modificadores de mundo (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2014; XU; VERBRUGGE, 2025a).

5.5.4 Estrutura Lógica versus Representação Espacial

A Teoria dos Grafos elucida a separação ontológica entre a estrutura lógica e a representação geométrica: um grafo de progressão é uma entidade abstrata, topológica e combinatória, e não uma especificação espacial métrica (DORMANS, 2010; SUMMERVILLE et al., 2018). Seus vértices e arestas estipulam a conectividade e a causalidade entre elementos, mas não contêm informações a respeito de posições tridimensionais (x, y, z), comprimentos em metros, geometrias de paredes ou desníveis verticais (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2013, 2014).

Essa dissociação estabelece uma relação de mapeamento de um-para-muitos: um mesmo grafo de progressão admite infinitas configurações espaciais distintas no espaço

contínuo tridimensional, contanto que as adjacências e ordenações lógicas sejam satisfeitas (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2013). Todavia, nem todo grafo lógico abstrato é imediatamente realizável em uma grade ortogonal ou em um plano arquitetônico contínuo (LINDEN; LOPES; BIDARRA, 2013, 2014). Se um vértice do grafo contiver um número de arestas incidentes (grau) superior ao número físico de portas comportadas pela geometria do módulo arquitetônico, ou se a topologia for não planar no plano bidimensional, a instanciação física necessita de técnicas de planarização e expansão estrutural, tais como a introdução algorítmica de nós intermediários (corredores ramificados, bifurcações ou transições verticais entre múltiplos pavimentos), conforme formalizado por van der Linden et al. (LINDEN; LOPES; BIDARRA, 2013, 2014).

5.6 Representação de Layouts Espaciais 3D

5.6.1 Representação Modular de Ambientes

Para converter a especificação abstrata de um grafo em um ambiente interativo tridimensional, a literatura de PCG e os motores de jogos apoiam-se no paradigma da modelagem espacial modular (LINDEN; LOPES; BIDARRA, 2014; SHAKER; LIAPIS et al., 2016; XU; VERBRUGGE, 2025a). Nessa abordagem, o espaço não é concebido como uma malha estática singular, mas através da composição combinatória de unidades tridimensionais reutilizáveis e pré-autoradas (LINDEN; LOPES; BIDARRA, 2014; SHAKER; LIAPIS et al., 2016; XU; VERBRUGGE, 2025a).

As granularidades modulares típicas abrangem (LINDEN; LOPES; BIDARRA, 2014; SHAKER; LIAPIS et al., 2016; VIEIRA, 2025):

- **Grades regulares e *voxels*:** partição do espaço em células discretas uniformes em 2D ou 3D, atribuindo-se a cada célula um elemento primitivo de geometria (bloco sólido, piso, parede, abertura ou ar livre) (SHAKER; TOGELIUS; NELSON, 2016; SHAKER; LIAPIS et al., 2016; SUMMERVILLE et al., 2018);
- **Segmentos compostos (*chunks*):** subestruturas volumétricas que agrupam conjuntos pré-definidos de blocos e acessórios temáticos (LINDEN; LOPES; BIDARRA, 2014; SHAKER; LIAPIS et al., 2016);
- **Módulos de salas e *prefabs* 3D:** blocos arquitetônicos inteiros (como salas com dimensões estipuladas, corredores em L, junções em T e escadarias) e ativos pré-fabricados (*prefabs*) de objetos internos (LINDEN; LOPES; BIDARRA, 2014; XU; VERBRUGGE, 2025a; REGMI, 2025). Um *prefab* encapsula a malha gráfica 3D (*mesh*), materiais, texturas, colunas de suporte, volumes de colisão física e pontos de ancoragem semânticos em um único pacote modular (XU; VERBRUGGE, 2025a; REGMI, 2025).

O acoplamento topológico e geométrico entre módulos adjacentes é operacionalizado por meio de conectores ou portas (*connectors* / *doorways* / *sockets*) posicionados

estrategicamente nas extremidades dos blocos modulares (LINDEN; LOPES; BIDARRA, 2013, 2014; SHAKER; LIAPIS et al., 2016; SUN et al., 2025). Cada conector define uma posição métrica relativa em relação ao pivô do módulo, a dimensão da abertura e um vetor de orientação normal que especifica a direção para a qual a passagem se projeta no espaço (LINDEN; LOPES; BIDARRA, 2013; SHAKER; LIAPIS et al., 2016; SUN et al., 2025; XU; VERBRUGGE, 2025a).

5.6.2 Relações e Transformações Espaciais

A instanciação de peças modulares no espaço contínuo $\mathbb{R}^3$ fundamenta-se nas transformações afins da álgebra linear, articulando o referencial de coordenadas local de cada objeto e o referencial global do cenário (*world space*) (ERICSON, 2004; SUN et al., 2025; XU; VERBRUGGE, 2025b).

A especificação espacial completa de um componente i exige distinguir sua *pose*, suas dimensões físicas e seus fatores de escala (ERICSON, 2004; SUN et al., 2025; XU; VERBRUGGE, 2025b):

- A **pose** define a localização e a atitude espacial do componente no mundo, decomposta na translação $\mathbf{p}_i = (x_i, y_i, z_i) \in \mathbb{R}^3$ (posição de seu ponto de pivô) e na rotação $\mathbf{R}_i \in SO(3)$ (orientação angular, expressa por matrizes ortogonais, quatérnios ou ângulos de Euler, tipicamente parametrizada em layouts pelo ângulo de guinada θ_i em torno do eixo vertical Y para rotações ortogonais de 90° ou 180°) (SUN et al., 2025; XU; VERBRUGGE, 2025a);
- As **dimensões nominais** $(w_i, l_i, h_i) \in \mathbb{R}^3$ representam a largura, o comprimento e a altura intrínsecos do volume do objeto no espaço local;
- O **fator de escala** $\mathbf{s}_i = (s_{x,i}, s_{y,i}, s_{z,i}) \in \mathbb{R}^3$ expressa a ampliação ou redução proporcional aplicada à geometria nos eixos coordenados.

Para a detecção de colisões e verificação de proximidade geométrica em tempo computacional eficiente, a computação gráfica recorre a volumes delimitadores (*bounding volumes*) (ERICSON, 2004; SUN et al., 2025; XU; VERBRUGGE, 2025b). Distinguem-se fundamentalmente dois tipos:

1. **Caixa Delimitadora Alinhada aos Eixos (*Axis-Aligned Bounding Box* — AABB):** definida pelos intervalos extremos $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}] \times [z_{\min}, z_{\max}]$ paralelamente aos eixos globais do sistema de coordenadas (ERICSON, 2004). Testes de interseção entre AABBs possuem custo computacional extremamente baixo por meio de comparações escalares de intervalos em cada eixo; contudo, quando um objeto sofre rotação angular arbitrária, sua AABB precisa ser recalculada e expandida para envolver os novos limites, introduzindo espaço vazio excessivo que pode gerar falsos positivos de colisão (ERICSON, 2004);

2. **Caixa Delimitadora Orientada (*Oriented Bounding Box* — OBB):** paralelepípedo delimitador centrado no objeto cujos eixos permanecem solidários à rotação e orientação da própria geometria local (ERICSON, 2004). Embora exija testes de separação mais sofisticados (como o Teorema dos Eixos Separadores — SAT), a OBB ajusta-se com precisão estrita ao volume sólido sob qualquer rotação espacial (ERICSON, 2004).

5.6.3 Restrições Geométricas

A montagem de um *layout* tridimensional coerente requer a satisfação concorrente de um conjunto estrito de restrições geométricas (LINDEN; LOPES; BIDARRA, 2014; ERICSON, 2004; SUN et al., 2025; XU; VERBRUGGE, 2025b,a):

- **Restrição de não sobreposição volumétrica (*non-overlap*):** sólidos arquitetônicos e objetos distintos não podem ocupar concomitantemente a mesma porção do espaço euclidiano tridimensional. Formalmente, a interseção volumétrica entre seus interiores sólidos deve ser estritamente nula ($V_{\text{overlap}} = 0$ ou $\text{IoU}_{3D} = 0$), evitando interpenetrações anômalas (*clipping*) (ERICSON, 2004; SUN et al., 2025; XU; VERBRUGGE, 2025b);

- **Restrição de contenção e fronteiras (*boundary constraints*):** objetos internos associados a uma sala devem ter seus volumes contidos estritamente no interior do polígono ou prisma formado por suas paredes, piso e teto, sem atravessar a alvenaria perimetral (SUN et al., 2025; XU; VERBRUGGE, 2025b);

- **Compatibilidade e alinhamento de portais:** para que duas salas ou uma sala e um corredor compartilhem uma passagem transitável, seus respectivos conectores devem coincidir métrica e volumetricamente na fronteira comum, apresentando vetores normais opostos e colineares, sem que ocorra bloqueio por paredes cegas de um dos módulos (LINDEN; LOPES; BIDARRA, 2013; SHAKER; LIAPIS et al., 2016; SUN et al., 2025; XU; VERBRUGGE, 2025a);

- **Desobstrução e espaço de circulação (*clearance*):** a distribuição de objetos em um cômodo deve resguardar zonas livres desimpedidas ao redor de portas, escadas e rotas centrais de passagem, prevenindo o congestionamento ou adensamento físico indevido (*overcrowding / clutter*) que impeça o trânsito físico dos agentes virtuais (SUN et al., 2025; XU; VERBRUGGE, 2025b,a).

5.7 Arquiteturas Híbridas e Geração Baseada em Restrições

5.7.1 Sistemas Híbridos e Neuro-Simbólicos

A análise aprofundada dos paradigmas generativos revela uma complementaridade ontológica: métodos puramente simbólicos, construtivos ou orientados a restrições fornecem

rigor matemático e preservação determinística de invariantes funcionais, porém demandam complexa formalização manual de regras e tendem a apresentar menor flexibilidade estilística (SHAKER; LIAPIS et al., 2016; SUMMERVILLE et al., 2018); inversamente, modelos baseados em redes profundas e LLMs exibem expressividade de linguagem, compreensão semântica e facilidade na geração de conteúdo criativo, porém sofrem de alucinações espaciais e ausência de garantias formais de consistência (GALLOTTA et al., 2024; MALEKI; ZHAO, 2024).

Essa constatação catalisou a emergência de *arquiteturas híbridas neuro-simbólicas* em inteligência artificial e geração de fases (GARCEZ; LAMB, 2023; GALLOTTA et al., 2024; XU; VERBRUGGE, 2025b). Conforme teorizado por Garcez e Lamb (GARCEZ; LAMB, 2023), o paradigma neuro-simbólico postula a integração sinérgica entre o aprendizado conexionista baseado em dados (capacidade de inferência intuitiva, correlação estatística e semântica de alto nível) e o raciocínio simbólico (estruturas lógicas, grafos, solucionadores e verificação formal baseada em regras). No contexto de PCG para ambientes 3D, essa divisão funcional traduz-se na delegação das etapas de planejamento conceitual, estruturação lógica e contextualização ao módulo de linguagem (neural), ao passo que as rotinas de resolução métrica contínua, instanciação modular e validação geométrica são executadas por algoritmos determinísticos e solucionadores formais (simbólicos) (HU et al., 2024; TANG et al., 2025; CHEN; JHALA, 2025; XU; VERBRUGGE, 2025b,a; HER et al., 2025).

De modo análogo, essa divisão conceitual dialoga com os princípios da geração de iniciativa mista (*Mixed-Initiative PCG*), fundamentada seminalmente em ferramentas como *Tanagra* (SMITH; WHITEHEAD; MATEAS, 2010) e *Sentient Sketchbook* (LIAPIS; YANNAKAKIS; TOGELIUS, 2013), nas quais o sistema computacional assume o papel de parceiro proativo que calcula métricas espaciais em tempo de autoria, resolve restrições e sugere correções estruturais mantendo o controle deliberativo do designer (SMITH; WHITEHEAD; MATEAS, 2010; LIAPIS; YANNAKAKIS; TOGELIUS, 2013; LAI; LATHAM; LEYMARIE, 2020; WALTON; RAHAT; STOVOLD, 2022; WANG et al., 2026).

5.7.2 Geração Baseada em Regras e Restrições

O suporte matemático para a imposição de regras em sistemas generativos reside na teoria de Problemas de Satisfação de Restrições (CSP — *Constraint Satisfaction Problem*) (RUSSELL; NORVIG, 2021). Formalmente, um CSP é definido pela tupla (X, D, C), na qual $X = \{X_1, X_2, \ldots, X_n\}$ denota um conjunto de variáveis de decisão (como coordenadas espaciais, orientações ou tipos de sala), $D = \{D_1, D_2, \ldots, D_n\}$ representa os domínios de valores admissíveis para cada variável, e $C = \{C_1, C_2, \ldots, C_m\}$ constitui um conjunto de restrições que delimitam as combinações simultâneas de valores permitidas (RUSSELL; NORVIG, 2021). A resolução de um CSP emprega algoritmos de retrocesso sistemático (*backtracking*) potencializados por heurísticas de ordenação e mecanismos de propagação de restrições, a exemplo da consistência de arcos (*arc consistency*), que elimina

antecipadamente valores de domínios que comprovadamente violariam condições futuras (RUSSELL; NORVIG, 2021).

No escopo do arranjo espacial e planejamento de fases, as restrições são categorizadas em duas ordens de rigor (SMITH; WHITEHEAD; MATEAS, 2010; XU; VERBRUGGE, 2025b,a):

- **Restrições rígidas (*hard constraints*):** condições mandatórias e invioláveis para a viabilidade do nível, cuja inobservância inviabiliza funcional ou fisicamente o artefato (tais como ausência de sobreposição sólida, contenção no perímetro e ordenação causal acíclica de chaves e portas) (SMITH; WHITEHEAD; MATEAS, 2010; SUN et al., 2025; XU; VERBRUGGE, 2025b);
- **Restrições flexíveis (*soft constraints*):** diretrizes de qualidade, preferências estéticas ou métricas de balanceamento que expressam objetivos desejáveis (tais como distâncias médias de deslocamento, agrupamento estilístico de móveis ou dispersão de desafios), cujo atendimento é otimizado sem configurar critério binário de rejeição (XU; VERBRUGGE, 2025b,a).

Quando o problema de disposição espacial envolve a ponderação conjunta de restrições rígidas e flexíveis em domínios contínuos, ele é frequentemente modelado por analogia ao clássico Problema de *Layout* de Instalações (*Facility Layout Problem* — FLP) da pesquisa operacional (XU; VERBRUGGE, 2025b,a). Sob essa formulação, define-se uma função objetivo com penalidades ponderadas associadas a cada infração de distância ou posicionamento, buscando configurações que minimizem o custo global de violações (XU; VERBRUGGE, 2025b).

5.7.3 Validação e Reparo de Conteúdo Gerado

Dado que geradores conexionistas, estocásticos ou heurísticos não garantem a priori a total conformidade com restrições rígidas, a literatura formaliza quatro paradigmas teóricos para a garantia da correção do conteúdo (SHAKER; TOGELIUS; NELSON, 2016; SUMMERVILLE et al., 2018; MALEKI; ZHAO, 2024; XU; VERBRUGGE, 2025a):

1. **Geração e teste com filtragem (Rejeição integral):** o gerador sintetiza o candidato e o submete a uma bateria de verificações algorítmicas; constatada qualquer infração rígida, o candidato é sumariamente descartado e o procedimento reiniciado (SHAKER; TOGELIUS; NELSON, 2016; MALEKI; ZHAO, 2024);
2. **Construção com retrocesso (*Backtracking*):** o sistema aloca componentes de forma incremental; identificada a impossibilidade de satisfação futura das restrições no nó atual da árvore de busca, o algoritmo desfaz a última atribuição e explora caminhos alternativos (SMITH; WHITEHEAD; MATEAS, 2010; LINDEN; LOPES; BIDARRA, 2014; RUSSELL; NORVIG, 2021);

3. **Reparo pós-geração (*Post-generation repair*):** operadores algorítmicos locais atuam sobre o artefato gerado para solucionar violações pontuais sem descartá-lo, tais como transladar pequenos móveis em colisão, regenerar conectores desalinhados ou desobstruir passagens bloqueadas (SUMMERVILLE et al., 2018; SUN et al., 2025; XU; VERBRUGGE, 2025a);

4. **Refinamento iterativo em malha fechada (*Feedback loop*):** diagnósticos detalhados extraídos pelos validadores formais são formatados como mensagens estruturadas de erro e reinjetados no módulo generativo de alto nível, permitindo correções informadas em iterações subsequentes (HU et al., 2024; NASIR; JAMES; TOGELIUS, 2024; XU; VERBRUGGE, 2025b; HER et al., 2025).

5.8 Validade, Navegabilidade e Solvabilidade de Fases

A avaliação formal da correção de uma fase de jogo exige decompor rigorosamente o conceito genérico de "jogabilidade funcional" em quatro dimensões teóricas distintas: validade lógica, validade geométrica, navegabilidade física e solvabilidade global. Essas dimensões operam em planos de abstração distintos e não configuram sinônimos (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2013, 2014; SUN et al., 2025; XU; VERBRUGGE, 2025b).

5.8.1 Validade Lógica e Estrutural

A validade lógica e estrutural diz respeito à consistência relacional e causal do nível em sua representação topológica abstrata, desconsiderando dimensões métricas ou físicas do espaço tridimensional (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2013, 2014). Uma fase é considerada logicamente válida quando satisfaz as invariantes fundamentais do sistema de regras:

- Presença de nós de início e fim devidamente parametrizados;
- Conectividade do grafo subjacente, atestando que todas as salas e objetivos necessários integram a mesma componente topológica conexa (LINDEN; LOPES; BIDARRA, 2014; XU; VERBRUGGE, 2025a);
- Consistência e aciclicidade das dependências causais (G_{dep} é um DAG), assegurando que não existem bloqueios circulares insolúveis, tais como trancar o acesso à chave de uma porta atrás da própria porta trancada (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2014; XU; VERBRUGGE, 2025a).

5.8.2 Validade Geométrica

A validade geométrica refere-se à integridade física do ambiente quando seus módulos são instanciados no espaço contínuo tridimensional $\mathbb{R}^3$ (ERICSON, 2004; SUN et al., 2025;

XU; VERBRUGGE, 2025b; FENG et al., 2026). Um layout 3D possui validade geométrica se, e somente se:

- Nenhuma malha arquitetônica sólida (paredes, pisos ou mobílias densas) sobrepõe-se a outra com volume de interseção positivo ($V_{\text{overlap}} = 0$) (ERICSON, 2004; XU; VERBRUGGE, 2025b);
- Todos os elementos internos respeitam os limites envoltórios do compartimento que os abriga (SUN et al., 2025; XU; VERBRUGGE, 2025b);
- Todas as portas e conectores adjacentes previstos na topologia estão devidamente alinhados e desimpedidos por paredes estruturais (LINDEN; LOPES; BIDARRA, 2013; SUN et al., 2025; XU; VERBRUGGE, 2025a).

A verificação dessa dimensão em motores gráficos e sistemas computacionais apoia-se em estruturas espaciais de consulta (como AABBs, OBBs, octrees e algoritmos de varredura como *raycasting*) (SHAKER; LIAPIS et al., 2016; ERICSON, 2004; SUN et al., 2025). Sob a ótica teórica, essas estruturas operam como ferramentas determinísticas de detecção e consulta espacial, fornecendo subsídios para que algoritmos de verificação confirmem ou rejeitem a conformidade geométrica do layout (SHAKER; LIAPIS et al., 2016; XU; VERBRUGGE, 2025b).

5.8.3 Navegabilidade e Malhas de Navegação

A navegabilidade (*navigability* ou *traversability*) atesta se o ambiente tridimensional oferece superfícies contínuas e transponíveis sobre as quais um agente cinemático (jogador ou NPC) é fisicamente capaz de se locomover (SNOOK, 2000; SUN et al., 2025; XU; VERBRUGGE, 2025b,a). A navegabilidade é uma propriedade dinâmica e dependente dos atributos físicos do agente, tais como altura livre requerida, raio cilíndrico de colisão, inclinação máxima de superfícies e altura máxima de degraus superáveis (SNOOK, 2000; XU; VERBRUGGE, 2025b). Um nível pode exibir perfeita validade geométrica (ausência de colisão entre malhas), mas ser desprovido de navegabilidade caso um corredor seja mais estreito que o diâmetro do agente ou caso ocorra um desnível vertical desprovido de escada ou rampa de acesso (SUN et al., 2025; XU; VERBRUGGE, 2025a).

A representação computacional padrão para o espaço contínuo transitável em ambientes tridimensionais consiste na Malha de Navegação (*Navigation Mesh* — NavMesh), introduzida na literatura de jogos por Snook (SNOOK, 2000). A NavMesh discretiza as superfícies caminháveis (*walkable areas*) da geometria 3D decompondo-as em um conjunto de polígonos convexos bidimensionais interconectados no espaço tridimensional (SNOOK, 2000; REGMI, 2025). A convexidade de cada polígono assegura que qualquer trajetória linear traçada entre dois pontos pertencentes ao mesmo polígono permanece estritamente contida dentro da área caminhável, sem colidir com obstáculos (SNOOK, 2000). O cálculo de rotas em uma NavMesh opera sobre o *grafo dual de adjacência* formado pelos polígonos

convexos (em que os polígonos constituem vértices e as arestas compartilhadas constituem arestas do grafo dual), aplicando-se posteriormente algoritmos de suavização geométrica (como o *funnel algorithm* ou *string pulling*) para gerar caminhos esteticamente suaves e métricamente otimizados (SNOOK, 2000; XU; VERBRUGGE, 2025a).

5.8.4 Busca de Caminhos (Pathfinding)

A busca de caminhos (*pathfinding*) consiste no procedimento algorítmico responsável por determinar uma rota contínua de travessia entre uma posição inicial $\mathbf{p}_{\text{start}}$ e uma posição alvo $\mathbf{p}_{\text{target}}$ sobre o espaço transitável discretizado (como a NavMesh) (HART; NILSSON; RAPHAEL, 1968; SNOOK, 2000; SUN et al., 2025; XU; VERBRUGGE, 2025a).

O algoritmo A^* (*A-star*), formulado por Hart, Nilsson e Raphael (HART; NILSSON; RAPHAEL, 1968), é a solução canônica para busca informada de caminhos em grafos de navegação. O A^* avalia vértices candidatos por meio de uma função de custo heurístico:

$$f(n) = g(n) + h(n) \tag{2}$$

onde $g(n)$ corresponde ao custo exato do caminho já percorrido da origem até o nó n, e $h(n)$ denota a estimativa heurística de custo de n até o objetivo (HART; NILSSON; RAPHAEL, 1968). Uma propriedade fundamental demonstrada por Hart et al. (HART; NILSSON; RAPHAEL, 1968) reside no critério de admissibilidade: se a função heurística $h(n)$ for admissível — ou seja, se nunca superestimar a distância real mínima até o alvo ($h(n) \leq h^*(n)$, como ocorre com a distância euclidiana em linha reta no espaço contínuo) —, o algoritmo A^* tem garantia matemática de encontrar um caminho de custo ótimo (mínimo), caso exista ao menos uma rota transponível (HART; NILSSON; RAPHAEL, 1968).

No contexto da verificação algorítmica de fases, a execução de testes de A^* entre pontos críticos comprova que o espaço modelado mantém conectividade física e operacional real entre compartimentos distintos (SUN et al., 2025; XU; VERBRUGGE, 2025a).

5.8.5 Solvabilidade de uma Fase

A solvabilidade (*solvability*, *winnability* ou completabilidade funcional) constitui o critério avaliativo superior de uma fase: ela estipula a existência de pelo menos uma trajetória válida e realizável pelo jogador no espaço de estados e no espaço geométrico tridimensional, partindo da configuração inicial e atingindo a condição de vitória sem violar nenhuma restrição física ou mecânica do jogo (SMITH; WHITEHEAD; MATEAS, 2010; DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2014; SUMMERVILLE et al., 2018; NASIR; JAMES; TOGELIUS, 2024; XU; VERBRUGGE, 2025b).

A distinção teórica entre validade geométrica, navegabilidade e solvabilidade é indispensável porque nenhuma dessas condições é suficiente para assegurar as demais

(DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2014; SUMMERVILLE et al., 2018; XU; VERBRUGGE, 2025b):

1. **Validade geométrica sem navegabilidade:** dois recintos podem ser posicionados sem nenhuma colisão volumétrica entre suas paredes, mas separados por uma vala intransponível ou por uma rampa com inclinação superior ao limite de locomoção do agente, inviabilizando a transposição física (SUN et al., 2025; XU; VERBRUGGE, 2025a);

2. **Navegabilidade física sem solvabilidade dinâmica:** o algoritmo A^* pode computar com sucesso um caminho contínuo na NavMesh estática entre a entrada e a saída do nível; entretanto, se esse trajeto interceptar uma porta trancada cuja respectiva chave estiver situada em uma câmara cujo acesso físico dependa da prévia transposição dessa mesma porta, o nível apresenta navegabilidade métrica, porém é absolutamente insolúvel devido ao impasse lógico de progressão (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2013; XU; VERBRUGGE, 2025b);

3. **Validade lógica abstrata sem solvabilidade espacial:** um grafo de progressão pode ter todas as suas relações de precedência perfeitamente ordenadas no plano abstrato ($\tau(chave) < \tau(porta)$), mas o nível resultante ser insolúvel porque a instanciação geométrica 3D posicionou um obstáculo físico denso que bloqueou o corredor de acesso à sala da chave, rompendo a transitabilidade real da fase (LINDEN; LOPES; BIDARRA, 2014; XU; VERBRUGGE, 2025a).

Portanto, a solvabilidade plena em ambientes tridimensionais com progressão não linear requer que a alcançabilidade seja verificada simultaneamente sob duas dimensões acopladas: a validade causal no espaço de estados lógicos e a conectividade física contínua comprovada no espaço métrico navegável da fase (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2014; XU; VERBRUGGE, 2025b,a).

5.8.6 Síntese Conceitual e Perspectiva Metodológica

A articulação dos conceitos expostos ao longo deste capítulo evidencia que a síntese de fases tridimensionais complexas impõe um duplo desafio formal: requer, por um lado, alta capacidade semântica e contextual para conceber narrativas, estruturas de desafio e progressões lógicas que regulem o ritmo do jogo; e exige, por outro lado, rigor determinístico absoluto para satisfazer restrições geométricas volumétricas, assegurar alinhamentos modulares e garantir a transitabilidade da malha de navegação.

O reconhecimento das limitações intrínsecas dos modelos conexionistas na manipulação direta de coordenadas métricas, contraposto ao poder estruturante dos formalismos de grafos e da satisfação de restrições, consolida a fundamentação teórica que orienta as abordagens modernas de cocriação e automação em jogos. Esse arcabouço conceitual — alicerçado na estrita distinção entre organização lógica e realização espacial, e na decomposição analítica entre validade geométrica, navegabilidade e solvabilidade — estabelece

as premissas indispensáveis para a formulação dos modelos, representações intermediárias e rotinas determinísticas de instanciação que serão detalhados no capítulo de Metodologia.

6 Trabalhos Relacionados

Neste capítulo são apresentados os principais trabalhos relacionados à geração procedural de fases e ambientes virtuais, com ênfase nas abordagens que estruturam a progressão lógica por grafos e naquelas que empregam Modelos de Linguagem de Grande Porte (LLMs) na síntese de cenas e níveis de jogos. De acordo com o paradigma de geração e o domínio espacial em que atuam, os trabalhos investigados podem ser divididos em três categorias principais:

- **Geração procedural baseada em grafos, restrições e iniciativa mista:** abordagens algorítmicas clássicas que separam a estrutura lógica da missão de sua realização geométrica ou utilizam solucionadores formais em coautoria com o projetista, mas que não operam a partir de instruções abertas em linguagem natural.
- **Geração de fases e cenas 2D assistida por LLMs:** métodos que exploram modelos de linguagem, via ajuste fino ou aprendizado em contexto, para sintetizar níveis matriciais discretos ou cenas bidimensionais baseadas em blocos (*tiles*), combinando a geração textual a verificações algorítmicas no plano.
- **Geração de cenas e fases 3D assistida por LLMs:** abordagens voltadas ao espaço tridimensional contínuo ($\mathbb{R}^3$), abrangendo desde a síntese de cômodos internos para modelagem e renderização até a construção de níveis fundamentada em otimização de restrições e integração com motores de jogos (*game engines*).

A seguir serão apresentados os trabalhos pesquisados dentro dessas três categorias, destacando o funcionamento de cada método, suas contribuições e suas limitações em relação ao problema tratado neste trabalho.

6.1 Geração Procedural Baseada em Grafos, Restrições e Iniciativa Mista

Os trabalhos desta categoria estabeleceram a base teórica e algorítmica para controlar a estrutura de progressão de fases e assegurar sua validade funcional por meio de estruturas de grafos e solucionadores de restrições.

6.1.1 Gramáticas de Missão e Espaço

Dormans (DORMANS, 2010) propôs uma abordagem para a geração procedural de fases em jogos de ação e aventura fundamentada na separação explícita entre duas estruturas: a *missão* e o *espaço*. Nessa metodologia, a missão é gerada primeiramente por meio de uma gramática de grafos direcionados que reescreve símbolos abstratos em cadeias de tarefas, chaves e fechaduras (*lock-and-key puzzles*). Somente após a consolidação do grafo de missão, uma segunda etapa emprega uma gramática de formas (*shape grammar*) para construir a topologia física de salas e conexões capaz de acomodar a sequência

lógica estipulada. Essa dissociação garante que o nível gerado preserve uma estrutura de progressão coerente independentemente da variação geométrica do mapa.

Com o objetivo de ampliar o controle do projetista sobre esse processo, van der Linden, Lopes e Bidarra (LINDEN; LOPES; BIDARRA, 2013, 2014) desenvolveram um método no qual restrições de jogabilidade especificadas pelo autor parametrizam gramáticas geradoras de grafos de ações (*action graphs*), nos quais tarefas que compartilham o mesmo espaço são agrupadas em nós de ações colocalizadas (*co-located actions*). A conversão desse grafo abstrato em um nível do jogo *Dwarf Quest* é executada em quatro passos sequenciais: (i) atribuição de espaço, que converte os nós do grafo de ações em salas e as arestas em corredores (*hallways*); (ii) pré-processamento de *layout*, que converte o grafo em um grafo planar (sem cruzamento de arestas) e limita o grau de cada vértice a no máximo quatro conexões (portas) mediante a inserção de novos nós intermediários de sala; (iii) resolução de *layout*, que mapeia o grafo planar sobre uma grade ortogonal bidimensional; e (iv) pós-processamento, que comprime corredores excessivamente longos (ou insere salas adicionais naqueles que não podem ser comprimidos), instanciando por fim a geometria tridimensional das salas, portas, objetos e personagens (LINDEN; LOPES; BIDARRA, 2013).

Apesar de garantirem a coerência causal das missões geradas, ambos os métodos exigem que o projetista codifique e mantenha manualmente conjuntos complexos de regras formais de produção e reescrita. Conforme apontado pelos próprios autores em sua revisão da área (LINDEN; LOPES; BIDARRA, 2014), a especificação direta de gramáticas formais permanece pouco intuitiva para o fluxo criativo de *Level Design*, além de carecer de mecanismos flexíveis para traduzir descrições conceituais abertas diretamente em estruturas de fases tridimensionais validadas.

6.1.2 Ferramentas de Iniciativa Mista e Solucionadores de Restrições

Paralelamente às gramáticas construtivas, outra vertente buscou integrar algoritmos procedurais diretamente ao processo de autoria humana por meio de ferramentas de iniciativa mista (*mixed-initiative*). Smith, Whitehead e Mateas (SMITH; WHITEHEAD; MATEAS, 2010) apresentaram a ferramenta *Tanagra*, voltada ao projeto de fases de plataforma 2D baseadas em ritmo (*beats*). No *Tanagra*, o designer pode fixar plataformas manualmente no espaço ou ajustar a linha de tempo de ações, enquanto um planejador reativo (ABL) acoplado a um solucionador numérico de satisfação de restrições (biblioteca Choco) preenche o restante do nível. Caso uma modificação do usuário inviabilize a fase, o sistema executa retrocesso (*backtracking*) ou relaxamento de restrições posicionais para assegurar que exista uma trajetória física válida entre o início e o fim do cenário (SMITH; WHITEHEAD; MATEAS, 2010).

Em uma linha complementar voltada a mapas estratégicos, Liapis, Yannakakis e Togelius (LIAPIS; YANNAKAKIS; TOGELIUS, 2013) propuseram o *Sentient Sketchbook*, uma ferramenta de autoria assistida por computador na qual o projetista edita esboços

abstratos de baixa resolução (*sketches*) de um nível em grade. Em tempo real, o sistema verifica restrições de jogabilidade — atestando se todas as bases e recursos são mutuamente alcançáveis, calculando os caminhos navegáveis mínimos entre eles (com identificação de pontos de estrangulamento e becos sem saída) e simulando a dificuldade de exploração por meio de um algoritmo de preenchimento (*flood-fill*) —, além de empregar Algoritmos Genéticos de duas populações (*FI-2pop GA*) combinados à busca por novidade (*novelty search*) para sugerir variações de mapas que preservam a validade estrutural (LIAPIS; YANNAKAKIS; TOGELIUS, 2013). Estudos posteriores sobre a adoção dessas ferramentas na indústria, conduzidos por Lai, Latham e Leymarie (LAI; LATHAM; LEYMARIE, 2020) e por Walton, Rahat e Stovold (WALTON; RAHAT; STOVOLD, 2022), demonstraram que a retenção de controle pelo designer, a integração ao fluxo de trabalho existente e a garantia determinística de caminhos transitáveis entre entrada e saída são requisitos essenciais para a utilidade prática de sistemas procedurais. Contudo, as ferramentas clássicas de iniciativa mista operam sobre abstrações bidimensionais em grade e dependem da manipulação manual de geometria ou parâmetros numéricos, não contando com a capacidade semântica de interpretar especificações em linguagem natural.

6.2 Geração de Fases e Cenas 2D Assistida por LLMs

Com a consolidação dos Modelos de Linguagem de Grande Porte baseados na arquitetura *Transformer* (GALLOTTA et al., 2024; MALEKI; ZHAO, 2024), as primeiras investigações sobre a síntese estrutural de níveis concentraram-se em domínios bidimensionais discretos, nos quais o espaço do jogo pode ser codificado diretamente como sequências de caracteres ou matrizes de blocos (*tiles*).

No trabalho de Todd et al. (TODD et al., 2023), foi investigada a capacidade de modelos de linguagem pré-treinados (GPT-2 e GPT-3) atuarem como geradores diretos de fases para o jogo de quebra-cabeça bidimensional *Sokoban*. Para isso, os mapas foram representados como sequências textuais de blocos (*tiles*) em grade, submetendo-se os modelos a ajuste fino supervisionado (*fine-tuning*) sobre conjuntos de níveis preexistentes para amostrar novos quebra-cabeças em tempo de teste (TODD et al., 2023; GALLOTTA et al., 2024; NASIR; JAMES; TOGELIUS, 2024). Os resultados indicaram que, enquanto o modelo GPT-2 apresenta dificuldades significativas quando o conjunto de treinamento é reduzido, o modelo GPT-3 consegue acomodar melhor bases de dados limitadas (GALLOTTA et al., 2024). Não obstante, conforme apontado em análises subsequentes da literatura (NASIR; JAMES; TOGELIUS, 2024; XU; VERBRUGGE, 2025b), abordagens baseadas no ajuste fino de LLMs para geração direta exigem treinamento específico para cada conjunto de regras e permanecem suscetíveis à aleatoriedade e a alucinações estruturais inerentes à inferência puramente autorregressiva sem verificador acoplado.

Visando superar a fragilidade da geração direta em passo único e dispensar o ajuste fino específico por jogo, Nasir, James e Togelius (NASIR; JAMES; TOGELIUS, 2024) propuseram o sistema *Word2World*, um pipeline *zero-shot* que projeta mundos

bidimensionais jogáveis a partir de narrativas. No *Word2World*, um LLM inicialmente cria uma história e extrai descrições de personagens, conjuntos de blocos (*tilesets*), objetivos e tipos de células caminháveis ou interativas. Em seguida, o modelo gera o mapa 2D como uma matriz de caracteres alfanuméricos em duas etapas: primeiro a disposição ambiental do terreno e, posteriormente, a alocação de personagens e objetos interativos. Como a saída bruta do LLM frequentemente apresenta linhas de comprimentos desiguais ou duplicação indevida de personagens únicos, o sistema aplica verificações e correções algorítmicas determinísticas (*algorithmic fixes*), preenchendo linhas defasadas até o comprimento máximo e removendo instâncias duplicadas. Por fim, o mapa passa por rodadas iterativas de avaliação e retroalimentação que combinam um avaliador baseado em LLM e um agente de busca heurística A^* (*AStar*) para verificar a existência de caminhos transitáveis até todos os objetivos (NASIR; JAMES; TOGELIUS, 2024).

Em outra abordagem voltada a ambientes 2D guiados por histórias, Chen e Jhala (CHEN; JHALA, 2025) desenvolveram um pipeline de geração narrativa-para-cena (*Narrative-to-Scene*). O método segmenta narrativas curtas em três quadros temporais (*time frames*) e utiliza um LLM para extrair predicados espaciais qualitativos no formato de triplas `Objeto-Relação-Objeto` (por exemplo, "casa abaixo da árvore" ou "árvore à direita do barril"). Os objetos identificados são associados a ativos visuais do conjunto de dados *GameTileNet* por meio de incorporações semânticas sensíveis a *affordances*. A construção física da cena combina um algoritmo de Autômatos Celulares para gerar a camada de terreno com um conjunto de regras simbólicas para posicionar os objetos segundo as relações extraídas (CHEN; JHALA, 2025).

Os métodos apresentados nesta seção demonstram a viabilidade de combinar a extração semântica de LLMs com algoritmos clássicos de PCG (como Autômatos Celulares, reparo matricial e busca A^*). Todavia, todas essas propostas encontram-se restritas a grades bidimensionais discretas ou cenas 2D estáticas. Conforme ressalvado pelos próprios autores em (CHEN; JHALA, 2025), tais sistemas não modelam transformações afins no espaço contínuo $\mathbb{R}^3$ (posição, orientação e dimensão volumétrica), não tratam a detecção de colisão entre malhas tridimensionais nem realizam a validação de Malhas de Navegação (NavMesh) acoplada a dependências causais de chaves e fechaduras.

6.3 Geração de Cenas e Fases 3D Assistida por LLMs

A transição de grades bidimensionais para ambientes tridimensionais contínuos impõe desafios severos de ancoragem espacial (*spatial grounding*) e consistência geométrica (GALLOTTA et al., 2024; MALEKI; ZHAO, 2024). Os trabalhos recentes que aplicam LLMs ao domínio 3D dividem-se entre a síntese de cômodos internos para modelagem e renderização visual, a geração de fases baseada em otimização de restrições e bancos de dados, e a integração direta com ferramentas procedurais e motores de jogos.

6.3.1 Síntese de Cenas e Cômodos 3D para Modelagem e Renderização

Uma primeira linha de investigação em 3D concentra-se no arranjo espacial de objetos e mobílias no interior de cômodos individuais a partir de descrições textuais ou imagens de referência. Öcal et al. (ÖCAL et al., 2024) propuseram o *SceneTeller*, um pipeline em três estágios para geração de salas 3D. A partir de um *prompt* descrevendo a disposição desejada, o sistema utiliza um LLM por meio de aprendizado em contexto (*in-context learning*), apoiado em exemplares do conjunto de dados 3D-FRONT, para prever as caixas delimitadoras 3D e orientações dos objetos adotando uma notação baseada em folhas de estilo CSS. Com base nesse *layout* inicial, o sistema recupera modelos CAD tridimensionais de uma base de dados e ajusta uma representação de *3D Gaussian Splatting* para permitir a estilização visual da cena via texto (ÖCAL et al., 2024).

Entretanto, a predição direta de coordenadas numéricas pelo LLM frequentemente ocasiona sobreposições físicas e erros de alinhamento. Para mitigar esse problema em softwares de modelagem, Hu et al. (HU et al., 2024) desenvolveram o agente *SceneCraft*, que converte descrições textuais em scripts Python executáveis no modelador Blender. Em vez de emitir coordenadas diretamente, o *SceneCraft* primeiramente abstrai a cena como um grafo de relacionamentos espaciais (*blueprint*) e traduz essas relações em restrições numéricas codificadas em Python. Em seguida, o sistema renderiza imagens da cena sob múltiplos ângulos e utiliza um Modelo de Visão e Linguagem (GPT-V) para identificar falhas visuais e refinar iterativamente o script, armazenando funções recorrentes em uma biblioteca reutilizável (HU et al., 2024). Seguindo a mesma estratégia de síntese de código para o Blender, Yang et al. (YANG et al., 2026) apresentaram o *Code-as-Room*, um arcabouço multiagente em múltiplos estágios baseado em modelos multimodais (MLLMs) dotado de um módulo de memória entre estágios que interpreta imagens de plantas baixas em vista superior (*top-down view*) para extrair relações espaciais e sintetizar o código da geometria, materiais e iluminação do recinto.

Buscando evitar que o LLM realize cálculos geométricos finos sem verificação espacial explícita, Sun et al. (SUN et al., 2025) propuseram o *RoomPlanner*, estruturado sob um pipeline de raciocínio-ancoragem, arranjo e otimização. O sistema emprega uma hierarquia de agentes LLM: um agente de alto nível seleciona o tipo de cômodo e os objetos pertinentes ao contexto, enquanto um agente de baixo nível infere escalas compatíveis com dimensões reais e regras de posicionamento. Os ativos 3D são gerados e inicializados como nuvens de pontos e submetidos a uma etapa de otimização iterativa em malha fechada guiada por duas restrições explícitas: uma restrição de colisão (para evitar sobreposição entre objetos e paredes) e uma restrição de alcançabilidade que rasteriza o mapa 3D em uma grade 2D transitável e executa o algoritmo A^* a partir da porta de entrada até cada objeto, finalizando com uma etapa de renderização via *3D Gaussian Splatting* que consome cerca de 30 minutos por ambiente (SUN et al., 2025).

A necessidade de módulos dedicados à validação física é corroborada pelo estudo de Feng et al. (FENG et al., 2026). Ao avaliarem quantitativamente métodos que tratam a

geração de *layouts* 3D diretamente como linguagem (em formato textual tipo JSON gerado por LLMs, como o LayoutGPT), os autores demonstraram que tais abordagens preservam a coerência semântica (*Semantic: Yes*), mas falham na plausibilidade física e na precisão espacial (*Physical: No*). Devido à ausência de modelagem física na inferência textual, os *layouts* gerados diretamente por LLMs apresentam colisões severas entre objetos, violações dos limites perimetrais da sala, oclusões de passagem e objetos flutuantes; para contornar essa limitação em cenas internas, Feng et al. (FENG et al., 2026) propuseram o *LaviGen*, adaptando um modelo generativo de difusão 3D para operar de forma autorregressiva diretamente sobre representações geométricas nativas.

Os trabalhos desta subseção comprovam empiricamente as limitações de ancoragem espacial dos LLMs e a importância de submeter o arranjo 3D a restrições explícitas de colisão e alcançabilidade. Não obstante, todos esses métodos restringem-se à disposição de mobília em cômodos residenciais isolados (*single-room layouts*) ou à síntese de cenas gráficas estáticas para renderização *offline* (no Blender ou via *Gaussian Splatting*), apresentando elevado custo computacional e desconsiderando a composição modular de fases com múltiplas salas e corredores, a geração de Malhas de Navegação em motores de jogos e a lógica de progressão mecânica.

6.3.2 Geração de Fases 3D Baseada em Otimização de Restrições e Bancos de Dados

Voltando-se especificamente para a construção de fases jogáveis em 3D, Xu e Verbrugge (XU; VERBRUGGE, 2025b) propuseram um arcabouço que combina LLMs com otimização matemática baseada no Problema de *Layout* de Instalações (*Facility Layout Problem* — FLP). No método proposto, o LLM recebe uma descrição narrativa da fase e a traduz para uma linguagem de domínio específico inspirada na linguagem Scenic, denominada GLDL (*Game Level Description Language*). O script GLDL declara instalações pré-definidas e variáveis (*facilities*), suas dimensões de caixa delimitadora e funções de restrição básicas (como `PlaceByWall` e `PlaceInRange`) e relacionais (como `Near`, `Far`, `CanSee`, `Alignment` e `Focus`). Em seguida, a determinação das coordenadas e orientações 3D é formulada como um problema de otimização que minimiza uma função objetivo composta pela soma ponderada de penalidades de violação de restrições (sobreposição, saída dos limites da arena, proximidade e visibilidade), grau de agrupamento (*clustering*) e esparsidade. Os autores avaliaram três métodos de otimização sobre 10.000 configurações geradas — Têmpera Simulada (*Simulated Annealing* — SA), Algoritmo Genético (GA) e busca gulosa (*Greedy*) —, selecionando o SA pela maior estabilidade numérica e validando a jogabilidade das arenas resultantes (como um supermercado de 20m $\times$ 30m $\times$ 6m) por meio de simulações com agentes em ambiente Unity (XU; VERBRUGGE, 2025b).

Nos testes de estabilidade do LLM reportados no próprio estudo (XU; VERBRUGGE, 2025b) sobre 5.000 scripts gerados com o modelo GPT-4o-mini, observou-se que, embora as sentenças de definição de objetos alcancem alta consistência (taxa média de correspondência de 82,36%), as restrições relacionais exibem maior variabilidade, atingindo

taxa média de correspondência de 25,38%. Além disso, conforme analisado pelos autores em trabalho subsequente (XU; VERBRUGGE, 2025a), a abordagem baseada diretamente em *prompts* GLDL em tempo de execução restringe-se ao arranjo detalhado de espaços individuais (*single-room*) e não escala diretamente para ambientes extensos de múltiplos pavimentos com progressão de jogabilidade integrada.

Para contornar essas limitações, Xu e Verbrugge (XU; VERBRUGGE, 2025a) desenvolveram um arcabouço orientado a bancos de dados (*Database-Driven Framework*) para a geração de níveis 3D em múltiplos andares. Nessa segunda proposta, os autores substituíram as chamadas ao LLM em tempo de geração por uma etapa prévia e *offline*, na qual o modelo (GPT-4o) é utilizado para popular três bancos de dados reutilizáveis em JSON contendo definições de instalações (*Facility Database*), modelos de salas (*Room Template Database*) e componentes mecânicos (*Mechanic Components Database*, como chaves e portas de saída). No momento de gerar um nível, algoritmos clássicos realizam uma busca em profundidade gulosa (*greedy DFS*) para empilhar e conectar modelos de salas estabelecendo uma ordem topológica global τ, otimizam o *layout* interno de cada sala via Têmpera Simulada (implementando todas as restrições, inclusive colisão com paredes e entre instalações, como penalidades flexíveis de peso elevado) e alocam os componentes de chave e fechadura entre as salas respeitando a precedência $\tau(R_{\text{chave}}) < \tau(R_{\text{porta}})$. Por fim, como a otimização estocástica por penalidades pode deixar obstruções físicas próximas às portas, o sistema executa um reparo em duas fases: uma passagem geométrica *offline* baseada em preenchimento (*flood-fill*) que reposiciona minimamente instalações adaptáveis bloqueadoras e, posteriormente, uma validação no motor Unity na qual um agente autônomo percorre as salas na ordem topológica τ utilizando busca A^* sobre a NavMesh, tentando reposicionar e, caso o bloqueio persista após o tempo limite, removendo da cena as instalações adaptáveis obstruidoras (XU; VERBRUGGE, 2025a).

Embora os dois trabalhos de Xu e Verbrugge (XU; VERBRUGGE, 2025b,a) representem os avanços mais próximos ao escopo desta pesquisa na literatura recente, observa-se um compromisso estrutural entre ambos. Enquanto o primeiro (XU; VERBRUGGE, 2025b) permite traduzir descrições em linguagem natural sob demanda, mas restringe-se ao arranjo interno de um espaço único sem progressão de chaves e portas, o segundo (XU; VERBRUGGE, 2025a) alcança a geração de múltiplas salas e progressão chave-porta ao retirar o LLM da etapa de concepção estrutural da fase sob demanda, restringindo-o ao preenchimento prévio de bancos de dados estáticos. Além disso, como o posicionamento espacial em ambos depende de otimização estocástica (Têmpera Simulada) sobre funções de penalidade flexíveis (*soft constraints*), a ausência de sobreposição e a desobstrução de passagens não são garantidas construtivamente na instanciação modular, exigindo etapas corretivas de reposicionamento ou remoção destrutiva de objetos após a geração.

6.3.3 Controle de Ferramentas Procedurais e Integração com Motores de Jogos

Uma terceira vertente em ambientes 3D investiga o uso de agentes baseados em LLMs para configurar parâmetros de ferramentas procedurais complexas ou interagir diretamente com motores de jogos comerciais. Her et al. (HER et al., 2025) propuseram uma arquitetura multiagente livre de treinamento (*training-free*), fundamentada no padrão Ator-Crítico (*Actor-Critic*), para a geração *zero-shot* de mapas 3D. Dado um *prompt* em linguagem natural, o agente Ator sintetiza uma proposta inicial de parâmetros técnicos para uma ferramenta procedural, enquanto o agente Crítico audita essa configuração confrontando-a com a documentação algorítmica estática e exemplos de referência em um ciclo iterativo de diálogo. Embora a separação dialógica entre geração e crítica aumente a taxa de sucesso na configuração de parâmetros em comparação ao uso de um único agente LLM, a verificação realizada pelo agente Crítico ocorre exclusivamente no plano textual da documentação (HER et al., 2025): não há inspeção determinística da geometria 3D instanciada, verificação de caixas delimitadoras (AABBs), cálculo de NavMesh ou validação de solvabilidade de missões.

No contexto de integração direta com motores de jogos de larga escala, Tang et al. (TANG et al., 2025) apresentaram o *UnrealLLM*, um arcabouço multiagente que conecta descrições em linguagem natural ao sistema nativo de geração procedural da *Unreal Engine 5* (UE5). O sistema utiliza agentes especializados apoiados em uma base de conhecimento de nós e padrões PCG e uma biblioteca multimodal de ativos 3D para traduzir instruções textuais em uma linguagem de domínio específico baseada em JSON, convertida por um *plugin* em *blueprints* procedurais executáveis na UE5, além de incorporar curvas paramétricas (*splines*) geradas via *SketchAgent* para delimitar zonas e caminhos de distribuição geométrica. Apesar de demonstrar alta interatividade e escalabilidade na geração de paisagens naturais (florestas, montanhas, desertos) e vilarejos externos (*towns*) (TANG et al., 2025), o *UnrealLLM* não aborda a construção modular de fases internas ou masmorras baseadas em salas e corredores conectados por portas, nem realiza a modelagem e validação formal de grafos de progressão com pré-condições lógicas de chaves e fechaduras.

De forma complementar, Regmi (REGMI, 2025) desenvolveu um sistema integrado ao motor Unity para a geração procedural de atividades de agentes virtuais baseada na interpretação semântica de cenas 3D por LLMs (avaliando modelos como ChatGPT, Gemini, Claude e Grok). Nesse sistema, contudo, o ambiente tridimensional e os objetos precisam ser posicionados manualmente pelo usuário no editor via operações de arrastar e soltar (*drag-and-drop*); o papel do LLM restringe-se a ler os metadados semânticos da cena já montada e gerar sequências estruturadas de ações para que os personagens caminhem (via `NavMeshAgent`) e interajam com os objetos (REGMI, 2025), não havendo geração procedural do *layout* espacial da fase.

As dificuldades práticas enfrentadas por desenvolvedores ao utilizar ferramentas generativas sem garantias formais de estrutura e geometria foram documentadas por

Bandeali (BANDEALI, 2026) e por Wang et al. (WANG et al., 2026). Por meio de uma revisão de escopo e de um estudo autoetnográfico sobre o fluxo de trabalho de *Level Design*, Bandeali (BANDEALI, 2026) constatou que ferramentas generativas 3D desprovidas de restrições operam frequentemente como "máquinas caça-níqueis" (*slot machines*) voláteis, produzindo ativos 3D defeituosos que forçam o desenvolvedor a um custoso processo de reparo manual, concluindo que a viabilidade prática exige arquiteturas pautadas em curadoria, restrições e reparo (*Curation, Constraint and Repair*). Na mesma direção, Wang et al. (WANG et al., 2026) demonstraram, por meio do sistema *DesignerlyLoop*, que a geração opaca em "caixa-preta" prejudica o alinhamento entre a intenção do autor e a saída do LLM, evidenciando o benefício de expor e curar a cadeia de raciocínio intermediária em uma estrutura diagramática de duas camadas antes da consolidação do resultado final.

6.4 Síntese Comparativa e Posicionamento do Trabalho

A Tabela 1 sintetiza as características técnicas dos principais trabalhos analisados neste capítulo em comparação com a metodologia proposta neste Trabalho de Conclusão de Curso, considerando o domínio espacial, o papel desempenhado pelo LLM, a representação intermediária adotada, o mecanismo de resolução espacial e prevenção de colisão, a validação de navegabilidade e solvabilidade lógica, e a integração com motores de jogos.

Como observado ao longo deste capítulo, os métodos clássicos baseados em gramáticas de grafos e restrições (DORMANS, 2010; LINDEN; LOPES; BIDARRA, 2013; SMITH; WHITEHEAD; MATEAS, 2010) oferecem garantias consistentes sobre a progressão lógica e a conectividade das fases, mas carecem da flexibilidade de interpretação semântica proporcionada pelos modelos de linguagem natural. Por outro lado, nos trabalhos recentes que aplicam LLMs à geração 3D, observa-se que as abordagens de síntese de cenas internas (ÖCAL et al., 2024; HU et al., 2024; SUN et al., 2025) restringem-se a cômodos isolados e processos custosos de renderização *offline*, enquanto as propostas voltadas a níveis de jogos ou limitam a geração guiada por *prompt* a uma única sala (XU; VERBRUGGE, 2025b), ou retiram o LLM do ciclo de planejamento estrutural da fase sob demanda para utilizá-lo apenas no preenchimento prévio de bancos de dados (XU; VERBRUGGE, 2025a), ou ainda focam na composição de cenários abertos sem progressão de chaves e portas (TANG et al., 2025).

A metodologia desenvolvida neste trabalho insere-se na interseção dessas abordagens ao propor uma arquitetura híbrida integrada como ferramenta (*addon*) no motor Godot Engine. Em vez de utilizar o LLM para prever coordenadas tridimensionais contínuas ou restringi-lo à criação *offline* de tabelas estáticas, o modelo de linguagem é empregado diretamente na interpretação do *prompt* do usuário para sintetizar a estrutura lógica da missão na forma de um grafo de progressão serializado em JSON (contendo salas, conexões e relações de dependência entre chaves e portas). A verificação formal das regras topológicas do grafo, o cálculo determinístico das transformações espaciais livres de sobreposição para a instanciação de módulos 3D e os testes de integridade da Malha de

Tabela 1 – Comparação entre os principais trabalhos relacionados e a abordagem proposta neste trabalho.

Trabalho	**Domínio**	**Papel do LLM**	**Representação**	**Resolução Espacial / Colisão**	**Navegabilidade e Solvabilidade**	**Motor de Jogo**
Dormans (DORMANS, 2010)	2D	Não utiliza	Grafo de missão	Gramática de formas	Precedência na gramática	Não
van der Linden et al. (LINDEN; LOPES; BIDARRA, 2013)	Grade 2D / 3D	Não utiliza	Grafo de ações	Planarização em grade 2D	Precedência no grafo de ações	Não
Smith et al. (SMITH; WHITEHEAD; MATEAS, 2010)	2D	Não utiliza	Linha de ritmo (*beats*)	Solver numérico (Choco)	Geometria jogável garantida	Não
Liapis et al. (LIAPIS; YANNAKAKIS; TOGELIUS, 2013)	2D	Não utiliza	Esboço em grade	Grade discreta	Caminhos mínimos e *flood-fill*	Não
Todd et al. (TODD et al., 2023)	2D	Gerador direto (*fine-tuning*)	Sequência de *tiles* ASCII	Sem verificação explícita	Não garantida	Não
Nasir et al. (NASIR; JAMES; TOGELIUS, 2024)	2D	Gerador e avaliador (*zero-shot*)	Matriz alfanumérica	Reparo algorítmico de linhas/células	Agente A^* (*AStar*) em grade 2D	Não
Chen e Jhala (CHEN; JHALA, 2025)	2D	Extrator de triplas	Triplas relacionais	Autômatos Celulares + regras	Não modela navegação/estados	Não
Öcal et al. (ÖCAL et al., 2024)	3D (Cômodo)	Gerador de *layout* (ICL)	Caixas em sintaxe CSS	Predição direta do LLM	Não aplicável (cena estática)	Não
Hu et al. (HU et al., 2024)	3D (Cena)	Sintetizador de código	Grafo + script Python	Restrições + *feedback* VLM	Não aplicável (render *offline*)	Não (Blender)
Yang et al. (YANG et al., 2026)	3D (Cômodo)	Agentes de código (MLLM)	Script Python	Regras por estágio no código	Não aplicável (render *offline*)	Não (Blender)
Sun et al. (SUN et al., 2025)	3D (Cômodo)	Planejador hierárquico	Regras + nuvem de pontos	Otimização de colisão	Alcance A^* no cômodo isolado	Não
Xu e Verbrugge (XU; VERBRUGGE, 2025b)	3D (Sala única)	Tradutor para GLDL	Script GLDL	FLP via SA (penalidades)	Simulação em Unity (sem chaves)	Sim (Unity)
Xu e Verbrugge (XU; VERBRUGGE, 2025a)	3D (Multiandar)	Apenas *offline* (bancos JSON)	Bancos de dados JSON	FLP via SA + reparo *flood-fill*	Ordem τ + NavMesh A^* (reparo)	Sim (Unity)
Her et al. (HER et al., 2025)	3D (Mapa)	Ator-Crítico de parâmetros	Configuração de parâmetros	Delegada à ferramenta PCG	Crítica textual da API (sem NavMesh)	Não
Tang et al. (TANG et al., 2025)	3D (Externo)	Agentes de *blueprint* PCG	DSL em JSON + *splines*	Sistema PCG nativo da UE5	Locomoção na UE5 (sem chaves)	Sim (UE5)
Regmi (REGMI, 2025)	3D (Cena fixa)	Planejador de ações de NPCs	Sequência codificada	Cena montada manualmente	`NavMeshAgent` para NPCs	Sim (Unity)
Proposta deste TCC	**3D (Modular)**	**Planejador lógico (JSON)**	**Grafo de progressão**	**Instanciação modular determinística**	**Validação de DAG + NavMesh e A^***	**Sim (Godot)**

Fonte: Elaborado pelo autor (2026).

Navegação (NavMesh) e de solvabilidade ficam sob responsabilidade exclusiva de algoritmos determinísticos executados no próprio motor de jogo.

Bibliografia

ARAÚJO, W. O. d.; ARANHA, E. H. d. S.; MADEIRA, C. A. G. Geração Procedural de Conteúdo Aplicada a Jogos Digitais Educacionais. In: ANAIS da VII Jornada de Atualização em Informática na Educação (JAIE 2018). Porto Alegre: Sociedade Brasileira de Computação, 2018. P. 1–20.

BANDEALI, A. **Generative AI & The Level Designer: An Exploratory Report of Tools, Workflows, and Challenges**. 2026. Master of Digital Media Major Research Project – Toronto Metropolitan University, Toronto, ON, Canada.

BENGIO, Y.; SIMARD, P.; FRASCONI, P. Learning Long-Term Dependencies with Gradient Descent Is Difficult. **IEEE Transactions on Neural Networks**, v. 5, n. 2, p. 157–166, 1994. DOI: 10.1109/72.279181.

BROWN, T. et al. Language Models are Few-Shot Learners. In: ADVANCES in Neural Information Processing Systems 33 (NeurIPS 2020). [S.l.: s.n.], 2020. v. 33, p. 1877–1901. Disponível em: <https://arxiv.org/abs/2005.14165>.

CHEN, Y.-C.; JHALA, A. Narrative-to-Scene Generation: An LLM-Driven Pipeline for 2D Game Environments. In: PROCEEDINGS of the AIIDE Workshop on Experimental AI in Games (EXAG). [S.l.: s.n.], 2025. arXiv: 2509.04481 [cs.GR]. Disponível em: <https://arxiv.org/abs/2509.04481>.

COHN, A. G.; RENZ, J. Qualitative Spatial Representation and Reasoning. In: HARMELEN, F. van; LIFSCHITZ, V.; PORTER, B. (Ed.). **Handbook of Knowledge Representation**. [S.l.]: Elsevier, 2008. P. 413–465. DOI: 10.1016/S1574-6526(07)03013-3.

CORMEN, T. H. et al. **Introduction to Algorithms**. 3. ed. Cambridge, MA, USA: MIT Press, 2009.

DORMANS, J. Adventures in level design: generating missions and spaces for action adventure games. In: PROCEEDINGS of the 2010 Workshop on Procedural Content Generation in Games. Monterey, California: Association for Computing Machinery, 2010. (PCGames '10), p. 1–8. ISBN 9781450300230. DOI: 10.1145/1814256.1814257. Disponível em: <https://doi.org/10.1145/1814256.1814257>.

ERICSON, C. **Real-Time Collision Detection**. San Francisco, CA, USA: Morgan Kaufmann, 2004. DOI: 10.1201/b14581.

FENG, H. et al. **Repurposing 3D Generative Model for Autoregressive Layout Generation**. [S.l.: s.n.], 2026. arXiv: 2604.16299 [cs.CV]. Disponível em: <https://arxiv.org/abs/2604.16299>.

GALLOTTA, R. et al. Large Language Models and Games: A Survey and Roadmap. **IEEE Transactions on Games**, Institute of Electrical e Electronics Engineers (IEEE), p. 1–18, 2024. ISSN 2475-1510. DOI: 10.1109/tg.2024.3461510. Disponível em: <http://dx.doi.org/10.1109/TG.2024.3461510>.

GARCEZ, A. d.; LAMB, L. C. Neurosymbolic AI: The 3rd Wave. **Artificial Intelligence Review**, v. 56, p. 12387–12406, 2023. DOI: 10.1007/s10462-023-10448-w.

GIBSON, J. J. **The Ecological Approach to Visual Perception**. Boston, MA, USA: Houghton Mifflin, 1979.

HARNAD, S. The Symbol Grounding Problem. **Physica D: Nonlinear Phenomena**, v. 42, n. 1–3, p. 335–346, 1990. DOI: 10.1016/0167-2789(90)90087-6.

HART, P. E.; NILSSON, N. J.; RAPHAEL, B. A Formal Basis for the Heuristic Determination of Minimum Cost Paths. **IEEE Transactions on Systems Science and Cybernetics**, v. 4, n. 2, p. 100–107, 1968. DOI: 10.1109/TSSC.1968.300136.

HER, L. C. et al. **Zero-shot 3D Map Generation with LLM Agents: A Dual-Agent Architecture for Procedural Content Generation**. [S.l.: s.n.], 2025. arXiv: 2512.10501 [cs.AI]. Disponível em: <https://arxiv.org/abs/2512.10501>.

HU, Z. et al. SceneCraft: An LLM Agent for Synthesizing 3D Scene as Blender Code. In: PROCEEDINGS of the 41st International Conference on Machine Learning (ICML). [S.l.: s.n.], 2024. arXiv: 2403.01248 [cs.CV]. Disponível em: <https://arxiv.org/abs/2403.01248>.

KARTH, I.; SMITH, A. M. WaveFunctionCollapse is Constraint Solving in the Wild. In: PROCEEDINGS of the 12th International Conference on the Foundations of Digital Games (FDG '17). New York, NY, USA: ACM, 2017. P. 1–10. DOI: 10.1145/3102071.3110566.

KHALIFA, A. et al. PCGRL: Procedural Content Generation via Reinforcement Learning. In: 1. PROCEEDINGS of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE). [S.l.: s.n.], 2020. v. 16, p. 95–101. DOI: 10.1609/aiide.v16i1.7416.

LAI, G.; LATHAM, W.; LEYMARIE, F. F. Towards Friendly Mixed Initiative Procedural Content Generation: Three Pillars of Industry. In: PROCEEDINGS of the International Conference on the Foundations of Digital Games (FDG '20). New York, NY, USA: Association for Computing Machinery, 2020. P. 1–4. DOI: 10.1145/3402942.3402946.

LIAPIS, A.; YANNAKAKIS, G. N.; TOGELIUS, J. Sentient Sketchbook: Computer-Aided Game Level Authoring. In: PROCEEDINGS of the 8th International Conference on the Foundations of Digital Games (FDG '13). [S.l.]: ACM, 2013. P. 213–220.

LINDEN, R. van der; LOPES, R.; BIDARRA, R. Designing Procedurally Generated Levels. In: PROCEEDINGS of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE Workshop). [S.l.]: AAAI Press, out. 2013. P. 41–47. DOI: 10.1609/aiide.v9i3.12592.

_____. Procedural Generation of Dungeons. **IEEE Transactions on Computational Intelligence and AI in Games**, v. 6, n. 1, p. 78–89, 2014. DOI: 10.1109/TCIAIG.2013.2290371.

MALEKI, M. F.; ZHAO, R. Procedural Content Generation in Games: A Survey with Insights on Emerging LLM Integration. In: 1. PROCEEDINGS of the Twentieth AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE). [S.l.]: AAAI Press, 2024. v. 20, p. 167–178. arXiv: 2410.15644 [cs.AI]. Disponível em: <https://arxiv.org/abs/2410.15644>.

NASIR, M. U.; JAMES, S.; TOGELIUS, J. **Word2World: Generating Stories and Worlds through Large Language Models**. [S.l.: s.n.], 2024. arXiv: 2405.06686 [cs.CL]. Disponível em: <https://arxiv.org/abs/2405.06686>.

ÖCAL, B. M. et al. SceneTeller: Language-to-3D Scene Generation. In: EUROPEAN Conference on Computer Vision (ECCV). Cham: Springer, 2024. P. 362–378.

REGMI, V. **A System for Procedural Generation of Virtual Agent Activity Using LLM-Based Scene Interpretation**. 2025. Master of Science Thesis – Purdue University, West Lafayette, IN, USA.

RUSSELL, S.; NORVIG, P. **Artificial Intelligence: A Modern Approach**. 4. ed. Hoboken, NJ, USA: Pearson, 2021.

SHAKER, N.; LIAPIS, A. et al. Constructive generation methods for dungeons and levels. In: PROCEDURAL Content Generation in Games: A Textbook and an Overview of Current Research. Cham: Springer, 2016. P. 31–55. DOI: 10.1007/978-3-319-42716-4_3.

SHAKER, N.; TOGELIUS, J.; NELSON, M. J. **Procedural Content Generation in Games: A Textbook and an Overview of Current Research**. Cham: Springer, 2016. DOI: 10.1007/978-3-319-42716-4.

SMITH, A. M.; MATEAS, M. Answer Set Programming for Procedural Content Generation: A Survey. **IEEE Transactions on Computational Intelligence and AI in Games**, v. 3, n. 3, p. 187–200, 2011. DOI: 10.1109/TCIAIG.2011.2158545.

SMITH, G.; WHITEHEAD, J. Analyzing the Expressive Range of a Level Generator. In: PROCEEDINGS of the Workshop on Procedural Content Generation in Games (PCGames '10). New York, NY, USA: ACM, 2010. P. 1–7. DOI: 10.1145/1814256.1814260.

SMITH, G.; WHITEHEAD, J.; MATEAS, M. Tanagra: a mixed-initiative level design tool. In: PROCEEDINGS of the Fifth International Conference on the Foundations of Digital Games. Monterey, California: Association for Computing Machinery, 2010. (FDG '10), p. 209–216. ISBN 9781605589374. DOI: 10.1145/1822348.1822376. Disponível em: <https://doi.org/10.1145/1822348.1822376>.

SNOOK, G. Simplified 3D Movement and Pathfinding Using Navigation Meshes. In: DELOURA, M. (Ed.). **Game Programming Gems**. Hingham, MA, USA: Charles River Media, 2000. P. 288–304.

SUMMERVILLE, A. et al. Procedural Content Generation via Machine Learning (PCGML). **IEEE Transactions on Games**, IEEE, v. 10, n. 3, p. 257–270, 2018. DOI: 10.1109/TG.2018.2846639. Disponível em: <https://doi.org/10.1109/TG.2018.2846639>.

SUN, W. et al. **RoomPlanner: Explicit Layout Planner for Easier LLM-Driven 3D Room Generation**. [S.l.: s.n.], 2025. arXiv: 2511.17048 [cs.CV]. Disponível em: <https://arxiv.org/abs/2511.17048>.

TANG, S. et al. UnrealLLM: Towards Highly Controllable and Interactable 3D Scene Generation by LLM-powered Procedural Content Generation. In_____. **Findings of the Association for Computational Linguistics: ACL 2025**. Vienna, Austria: Association for Computational Linguistics, jul. 2025. P. 19417–19435. DOI: 10.18653/v1/2025.findings-acl.994. Disponível em: <https://aclanthology.org/2025.findings-acl.994/>.

TODD, G. et al. Level Generation Through Large Language Models. In: PROCEEDINGS of the 18th International Conference on the Foundations of Digital Games. [S.l.]: ACM, abr. 2023. (FDG 2023), p. 1–8. DOI: 10.1145/3582437.3587211. Disponível em: <http://dx.doi.org/10.1145/3582437.3587211>.

TOGELIUS, J. et al. Search-Based Procedural Content Generation: A Taxonomy and Survey. **IEEE Transactions on Computational Intelligence and AI in Games**, v. 3, n. 3, p. 172–186, 2011. DOI: 10.1109/TCIAIG.2011.2148116.

TOTTEN, C. W. **An Architectural Approach to Level Design**. Boca Raton, FL, USA: CRC Press, 2014. DOI: 10.1201/b17025.

VASWANI, A. et al. Attention Is All You Need. In: ADVANCES in Neural Information Processing Systems 30 (NeurIPS 2017). [S.l.: s.n.], 2017. v. 30, p. 5998–6008. Disponível em: <https://arxiv.org/abs/1706.03762>.

VIEIRA, A. H. G. **Metodologia de geração procedural de conteúdo para estruturas narrativas em jogos eletrônicos**. Fev. 2025. Tese (Doutorado) – Universidade Estadual Paulista (Unesp), Bauru. Disponível em: <https://hdl.handle.net/11449/295661>.

WALTON, S. P.; RAHAT, A. A. M.; STOVOLD, J. Evaluating Mixed-Initiative Procedural Level Design Tools Using a Triple-Blind Mixed-Method User Study. **IEEE Transactions on Games**, Institute of Electrical e Electronics Engineers (IEEE), v. 14, n. 3, p. 413–422, set. 2022. ISSN 2475-1510. DOI: 10.1109/tg.2021.3086215. Disponível em: <http://dx.doi.org/10.1109/TG.2021.3086215>.

WANG, A. et al. DesignerlyLoop: Forming Design Intent through Curated Reasoning for Human-LLM Alignment. In: PROCEEDINGS of the Designing Interactive Systems Conference (DIS '26). New York, NY, USA: Association for Computing Machinery, 2026. P. 661–687. DOI: 10.1145/3800645.3812885. Disponível em: <https://doi.org/10.1145/3800645.3812885>.

XU, K.; VERBRUGGE, C. A Database-Driven Framework for 3D Level Generation with LLMs. In: PROCEEDINGS of the Twenty-First AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment (AIIDE). [S.l.]: AAAI Press, 2025. P. 367–376. arXiv: 2508.18533 [cs.AI]. Disponível em: <https://arxiv.org/abs/2508.18533>.

_____. Constraint Is All You Need: Optimization-Based 3D Level Generation with LLMs. In: PROCEEDINGS of the 20th International Conference on the Foundations of Digital Games. New York, NY, USA: Association for Computing Machinery, 2025. (FDG '25). DOI: 10.1145/3723498.3723840. Disponível em: <https://doi.org/10.1145/3723498.3723840>.

YANG, Y. et al. **Code-as-Room: Generating 3D Rooms from Top-Down View Images via Agentic Code Synthesis**. [S.l.: s.n.], 2026. arXiv: 2605.18451 [cs.CV]. Disponível em: <https://arxiv.org/abs/2605.18451>.